



TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN SISTEMAS DE INFORMACIÓN



Modernización del módulo Report Service en la plataforma BIC para el cliente AESLA

Estudiante: Andrea Liñares Castro

Dirección: Fernando Silva Coira
Ángel Fernández Formoso

A Coruña, septiembre de 2026.

A GBTEC Software AG, por la oportunidad de crecer y aprender en un entorno profesional.

Agradecimientos

Quiero agradecer a GBTEC Software AG, y especialmente a Ángel, mi tutor, y a Ana, mi compañera, la oportunidad de realizar este proyecto en un entorno profesional, así como la confianza, la orientación y el apoyo recibidos durante estos meses.

También agradezco a mi familia, a mis amigos y a mi pareja su apoyo constante, su paciencia y sus palabras de ánimo durante la elaboración de este Trabajo de Fin de Grado. Gracias igualmente a mis compañeros de clase por todos estos años de aprendizaje compartido, por la ayuda mutua y por los buenos momentos vividos durante la carrera.

Resumen

En el contexto de la gestión de procesos empresariales y la generación automática de informes ejecutivos, este Trabajo de Fin de Grado presenta la modernización del módulo Report Service para AESLA, desarrollado por GBTEC Software AG. El proyecto aborda la optimización del flujo de generación de informes en formato Excel, mediante la incorporación de almacenamiento histórico, procesamiento asíncrono, control de duplicidad y mejoras en la experiencia de usuario. Para ello, se aplicó una metodología ágil iterativa e incremental, utilizando tecnologías como Spring Batch, Flyway, PostgreSQL, Elasticsearch y Angular. Las mejoras contribuyen a aumentar la escalabilidad, robustez y calidad del servicio, alineándose con las necesidades operativas y analíticas de AESLA.

Abstract

Within the scope of business process management and automatic generation of executive reports, this undergraduate thesis presents the modernization of the Report Service module for AESLA, developed by GBTEC Software AG. The project focuses on optimizing the report generation workflow in Excel format by incorporating historical storage, asynchronous processing, duplication control, and user experience improvements. An iterative and incremental agile methodology was applied, employing technologies such as Spring Batch, Flyway, PostgreSQL, Elasticsearch, and Angular. These enhancements contribute to increased scalability, robustness, and service quality, aligning with AESLA's operational and analytical needs.

Palabras clave:

- Generación automática de informes
- Procesamiento asíncrono
- Metodología ágil
- Spring Batch
- PostgreSQL
- Elasticsearch
- Angular
- Control de duplicidad
- Persistencia de informes

Keywords:

- Automatic report generation
- Asynchronous processing
- Agile methodology
- Spring Batch
- PostgreSQL
- Elasticsearch
- Angular
- Duplication control
- Report persistence

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	6
2	Fundamentos tecnológicos	7
2.1	Estado del arte	7
2.2	Tecnologías utilizadas	9
2.2.1	Servidor web	9
2.2.2	Cliente web	9
2.2.3	Base de datos	10
2.2.4	Infraestructura y Despliegue	10
2.2.5	Herramientas de apoyo	10
3	Metodología y planificación	12
3.1	Metodología de desarrollo	12
3.1.1	Adaptación al proyecto	13
3.2	Planificación y seguimiento	13
3.2.1	Planificación inicial	14
3.2.2	Seguimiento real	14
4	Análisis	16
4.1	Requisitos	16
4.1.1	Actores y Sistemas involucrados	16
4.1.2	Requisitos funcionales	16
4.1.3	Requisitos no funcionales	17
4.1.4	Historias de usuario	18
4.2	Arquitectura del sistema	19
4.2.1	Cliente web	19

4.2.2	Servidor	20
4.2.3	Capa de datos	20
4.3	Interfaz de usuario	20
5	Diseño	22
5.1	Arquitectura tecnológica del sistema	22
5.1.1	Visión general de la arquitectura	22
5.1.2	Comunicación entre componentes	25
5.1.3	Justificación de la arquitectura elegida	26
5.1.4	Stack tecnológico	26
5.2	Diseño de la aplicación	27
5.2.1	Cliente	27
5.2.2	Servidor (Spring Boot + Spring Batch)	31
5.2.3	Capa de datos	36
5.3	Interacción cliente-servidor	38
5.3.1	Endpoints REST	38
5.3.2	Manejo de errores	40
6	Implementación y pruebas	41
6.1	Tareas de mejora	41
6.1.1	Organización de los work items	41
6.1.2	Persistencia y consulta de informes	42
6.1.3	Vulnerabilidades	42
6.1.4	Coverage	43
6.1.5	Refactorización	44
6.1.6	Duplicación	44
6.1.7	Generación asíncrona con Spring Batch	45
6.1.8	Observabilidad y soporte	45
6.1.9	Mejoras de la interfaz	46
6.1.10	Incidencias y trabajo no realizado	46
6.2	Pruebas	46
6.2.1	Relación entre requisitos y pruebas	46
6.2.2	Catálogo representativo de pruebas	47
6.2.3	Pruebas manuales y entorno de QA	48
7	Solución desarrollada	49
7.1	Introducción	49
7.2	Página de Generar	49

7.2.1	Funcionalidad	50
7.2.2	Diseño de la interfaz	50
7.2.3	Flujo de interacción	51
7.3	Página de Historial	52
7.3.1	Funcionalidad	52
7.3.2	Diseño de la interfaz	52
7.4	Release y publicación	53
8	Conclusión y trabajo futuro	55
8.1	Conclusiones	55
8.2	Limitaciones	56
8.3	Posibles ampliaciones futuras	57
8.3.1	Actualización de la versión de Angular	57
8.3.2	Mejoras en la gestión del historial	58
8.3.3	Refactorización adicional del código backend	59
8.3.4	Funcionalidades avanzadas	60
A	Evidencias técnicas	62
A.1	Arquitectura detallada	62
A.2	Evidencias de calidad de código	64
	Lista de acrónimos	67
	Glosario	69

Índice de figuras

1.1	Mapa de presencia internacional de AESLA (Food Service Project). Fuente: elaboración propia a partir de [?].	1
1.2	Contexto de la plataforma BIC BPM mostrando la arquitectura de sistemas y sus interacciones. Fuente: elaboración propia a partir de [?].	3
1.3	Ejemplo de un proceso BPMN sencillo modelado en BIC Process Design. Fuente: elaboración propia a partir de [?].	4
1.4	Informe de tipo KPI con categoría de proceso Alternativa (todas las franquicias, sin rango de fechas).	5
1.5	Informe de tipo Producto con categoría de proceso Innovación (todas las franquicias, sin rango de fechas).	5
3.1	Diagrama de Gantt de la planificación inicial del proyecto.	14
3.2	Diagrama de Gantt correspondiente al seguimiento real del proyecto.	15
4.1	Diagrama de casos de uso principales del Report Service. Fuente: elaboración propia.	19
4.2	Interfaz original del Report Service antes de su modernización.	21
5.1	Arquitectura simplificada del flujo principal de generación de informes. Fuente: elaboración propia.	24
7.1	Interfaz principal del módulo de generación de informes.	51
7.2	Diagrama de flujo del proceso de generación de informes.	51
7.3	Vista de la página de historial de informes generados.	53
8.1	Mockup de la interfaz de historial con funcionalidad de eliminación de informes.	58
8.2	Diálogo de confirmación para la eliminación de informes.	59
A.1	Arquitectura detallada del Report Service. Fuente: elaboración propia.	63

A.2	Captura de SonarCloud correspondiente a la cobertura inicial del proyecto: 17.7%.	64
A.3	Captura de SonarCloud correspondiente a la cobertura final del proyecto: 25.7%.	65
A.4	Captura de SonarCloud correspondiente a la duplicación inicial del proyecto: 15.8%.	65
A.5	Captura de SonarCloud correspondiente a la duplicación final del proyecto: 10.0%.	66

Índice de cuadros

2.1	Comparación cualitativa de soluciones relacionadas con el análisis y la generación de informes.	8
5.1	Componentes principales y responsabilidades del Report Service.	23
5.2	Atributos de la entidad <i>Report</i> en <i>aesla_reports.report</i>	38
6.1	Evolución de las principales métricas de calidad medidas con SonarCloud. . .	45
6.2	Trazabilidad entre requisitos funcionales y pruebas realizadas.	47
6.3	Casos de prueba representativos del Report Service.	47

Capítulo 1

Introducción

1.1 Motivación

AESLA, S.A.B. de C.V., es una empresa multinacional mexicana dedicada a la operación de marcas de restauración franquiciadas. Con presencia en América Latina y Europa, se ha consolidado como uno de los principales operadores del sector, gestionando cadenas tan conocidas como Starbucks, Burger King, Domino's Pizza, Vips o Foster's Hollywood [?]. En España, su actividad se articula a través de Food Service Project, que engloba establecimientos de restauración rápida, cafeterías y restaurantes de comida casual.¹



Figura 1.1: Mapa de presencia internacional de AESLA (Food Service Project). Fuente: elaboración propia a partir de [?].

La magnitud de esta operación internacional hace que AESLA maneje diariamente un volumen muy elevado de información relativa a ventas, operaciones, proyectos de innovación,

¹ Nota: El nombre del cliente ha sido modificado por motivos de confidencialidad.

indicadores de desempeño y actividades internas. En este contexto, disponer de datos fiables, centralizados y fácilmente accesibles resulta fundamental para evaluar el rendimiento de las franquicias, identificar posibles desviaciones y tomar decisiones estratégicas fundamentadas. Sin herramientas de análisis adecuadas, la gestión de este conocimiento se vuelve compleja y poco eficiente.

Para dar respuesta a esta necesidad, GBTEC Software AG, compañía alemana especializada en soluciones de gestión de procesos de negocio ([Business Process Management \(BPM\)](#)), arquitectura empresarial ([Enterprise Architecture Management \(EAM\)](#)) y gobierno, riesgo y cumplimiento ([Governance, Risk and Compliance \(GRC\)](#)), ofrece la *BIC Platform*, un ecosistema de software que permite modelar, ejecutar y analizar procesos empresariales [? ?]. Dentro de este marco, se desarrolló para AESLA un módulo específico denominado *Report Service*, cuya finalidad es transformar los datos de la plataforma de Process Execution en informes ejecutivos descargables en formato Excel.

La *BIC Platform* se despliega habitualmente en modalidad cloud (BIC Cloud) y está compuesta por varios módulos que cubren el ciclo de vida de los procesos: *Process Design* para modelar procesos mediante diagramas BPMN, y *Process Execution* para ejecutar y automatizar esos procesos, gestionando instancias, tareas y persistencia de variables [? ?]. El *Report Service* extrae información de *Process Execution* (combinada con consultas a [Elasticsearch](#)) para componer informes Excel con [KPI](#), [Retroplanning](#), [Heatmap](#) y otras métricas solicitadas por AESLA.

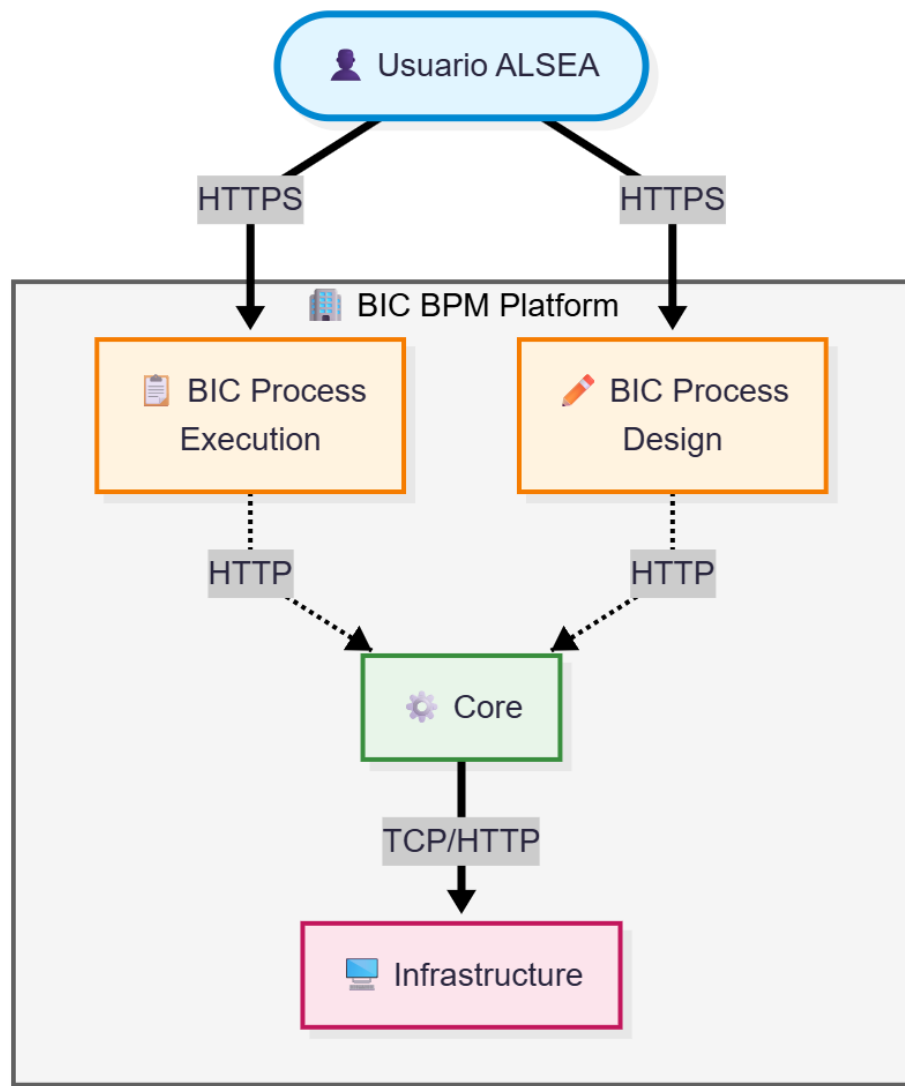


Figura 1.2: Contexto de la plataforma BIC BPM mostrando la arquitectura de sistemas y sus interacciones. Fuente: elaboración propia a partir de [?].

La figura 1.2 ilustra la arquitectura completa de la plataforma BIC BPM, mostrando cómo el usuario interactúa mediante *HTTPS* con los módulos *BIC Process Design* y *BIC Process Execution*, que a su vez se comunican con el núcleo (*Core*) y la capa de infraestructura (*Infrastructure*). Esta arquitectura de sistemas distribuidos proporciona la base sobre la que se integra el Report Service desarrollado en este proyecto.

Para facilitar la comprensión del papel de *BIC Process Design* dentro de la plataforma, la figura 1.3 muestra un ejemplo sencillo de un proceso de negocio modelado mediante BPMN. Este tipo de diagramas permite representar gráficamente el flujo de un proceso mediante

eventos, actividades y decisiones, proporcionando una visión estructurada de las distintas etapas que lo componen.

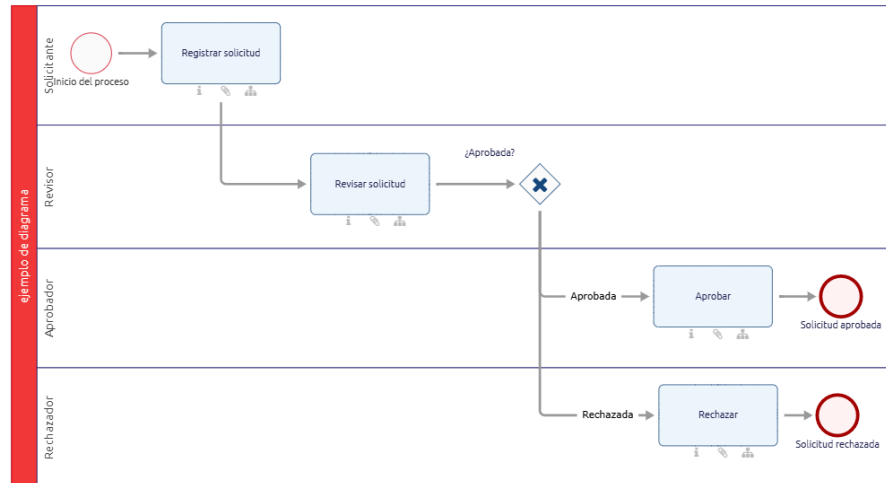


Figura 1.3: Ejemplo de un proceso BPMN sencillo modelado en BIC Process Design. Fuente: elaboración propia a partir de [?].

Los modelos definidos en *Process Design* sirven como representación de los procesos de negocio de la organización. Cuando estos procesos se trasladan al ámbito de ejecución, *Process Execution* permite gestionar sus instancias, tareas y variables asociadas. Parte de esta información constituye posteriormente la fuente de datos utilizada por el *Report Service* para elaborar los informes solicitados por AESLA.

El Report Service original ofrecía una solución sencilla y funcional: la interfaz permitía seleccionar el tipo de informe (KPI o Producto), la categoría (Innovación o Alternativa), una o varias franquicias y un rango de fechas. Con esta información, el backend recuperaba los datos correspondientes, generaba un libro de Excel con múltiples hojas y lo devolvía al usuario para su descarga local.

Cada combinación produce un libro Excel con múltiples hojas que cubren distintos requisitos y métricas personalizadas solicitadas por el cliente. A continuación se muestran dos ejemplos representativos que ilustran la estructura y contenido de estos informes:

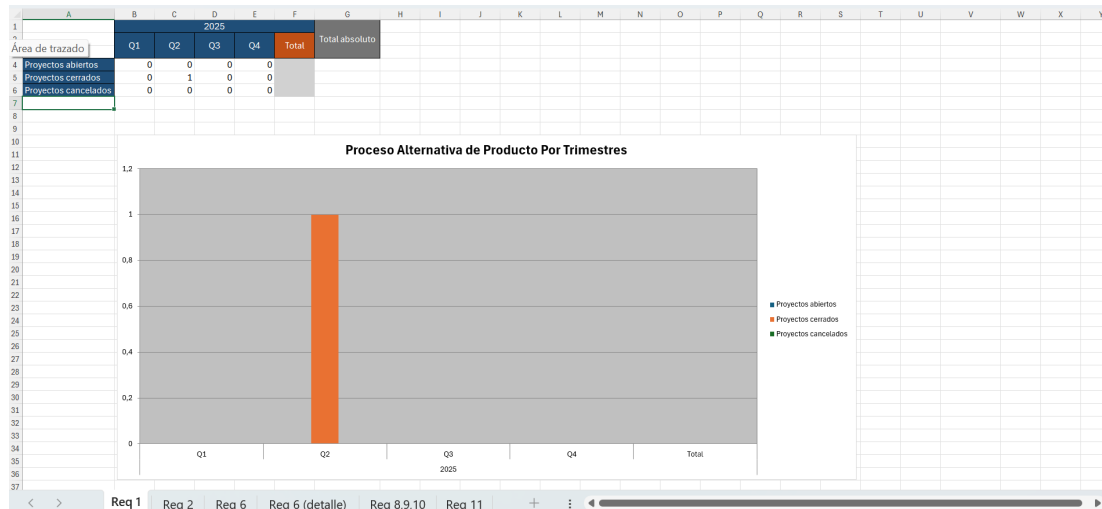


Figura 1.4: Informe de tipo KPI con categoría de proceso Alternativa (todas las franquicias, sin rango de fechas).

La anterior figura muestra la hoja del Requisito 1, que presenta un **Histograma** con los procesos de alternativa de producto por trimestres, diferenciando proyectos abiertos, cerrados y cancelados, además de los totales y el total absoluto en 2025. En la parte inferior se observan las pestañas de otras hojas del libro, correspondientes a requisitos adicionales personalizados solicitados por el cliente.

A	B	C	D	E	F	G	H	I
Marca	Estado	Proyecto	Actividad	Responsable	Fecha de comienzo inicial	Fecha de comienzo actual	Fecha de finalización inicial	Fecha de finalización actual
Foster's Hollywood	ACTIVO	test_ana_6	Trabajosdip	DDP	04-jun-25		19-jun-25	
Foster's Hollywood	ACTIVO	test_ana_6	DOP2	DDP	19-jun-25		03-ago-25	
Starbucks España	ACTIVO	test_bruno	Estimacionventa	MKT	25-may-25		01-jun-25	
Starbucks España	ACTIVO	test_bruno	Comunicacionventa	LOGISTICA	01-jun-25		08-jun-25	
Starbucks España	ACTIVO	test_bruno	Cálculoconsumo	GESTION DE PRODUCTO	08-jun-25		08-jun-25	
Starbucks España	ACTIVO	test_bruno	Formación	QA QA	08-jun-25		22-jun-25	
Starbucks España	ACTIVO	test_bruno	Productolmazen	LOGISTICA	22-jun-25		20-jul-25	
Starbucks Holand	ACTIVO	test_bruno_2	DOP2	DDP	19-abr-25		03-jun-25	
Starbucks Holand	ACTIVO	test_bruno_2	Muestras	GESTION DE PRODUCTO	03-jun-25		10-jun-25	
Starbucks Holand	ACTIVO	test_bruno_2	Catso	DDP	10-jun-25		17-jun-25	
Starbucks Holand	ACTIVO	test_bruno_2	DecisionMarca	MKT	17-jun-25		24-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Estimacionventa	MKT	24-jun-25		01-jul-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Focus	C_INTELIGENCE	14-may-25		04-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Maguetas	GESTION DE PRODUCTO	04-jun-25		11-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Catso	DDP	11-jun-25		19-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	DecisionMarca	MKT	19-jun-25		25-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Internacional	MKT	25-jun-25		02-jul-25	

Figura 1.5: Informe de tipo Producto con categoría de proceso Innovación (todas las franquicias, sin rango de fechas).

La anterior figura muestra la hoja del heatmap actual, que representa para cada marca el estado de sus proyectos junto con las actividades, responsables que las llevan a cabo y fechas previstas. En la parte inferior se pueden observar las pestañas de otras hojas del libro,

incluyendo los retroplannings y una hoja de comparativa, que forman parte del conjunto de métricas y análisis solicitados por AESLA.

Estos ejemplos muestran la riqueza informativa y el nivel de personalización de los informes generados por el Report Service, diseñados para cubrir las necesidades analíticas específicas de AESLA en la gestión de sus procesos de innovación y alternativa de producto.

Sin embargo, con el paso del tiempo surgieron limitaciones notables. El sistema carecía de un almacenamiento histórico de informes, lo que obligaba a los usuarios a regenerarlos cada vez que deseaban acceder a información pasada. Además, se detectaron problemas de duplicación de código en el backend, ausencia de procesamiento asíncrono (lo que generaba bloqueos en la interfaz durante la creación de los informes) y una experiencia de usuario que no reflejaba plenamente las necesidades de la organización. Estas carencias constituyeron la motivación principal para emprender un proceso de mejora y modernización del Report Service.

1.2 Objetivos

El presente Trabajo de Fin de Grado documenta, analiza y evalúa las mejoras introducidas en el Report Service de AESLA durante mis prácticas en GBTEC. En concreto, se plantearon los siguientes objetivos:

- Ampliar la funcionalidad del servicio con almacenamiento histórico en base de datos.
- Incrementar la mantenibilidad y calidad del código, reduciendo duplicaciones y aumentando la cobertura de pruebas.
- Optimizar la experiencia de usuario en la interfaz de generación de informes y acceso al historial.
- Introducir procesamiento asíncrono con [Spring Batch](#), evitando bloqueos en la interfaz.
- Añadir control de duplicidad para evitar regeneraciones innecesarias.
- Permitir la redescarga de informes desde el historial.

Con ello, se pretende transformar el Report Service en una solución completa que cubra tanto la generación como la gestión del ciclo de vida completo de los informes ejecutivos.

Fundamentos tecnológicos

2.1 Estado del arte

Para la elaboración del estado del arte se han analizado distintas soluciones disponibles en el mercado relacionadas con la generación de informes empresariales y cuadros de mando, evaluando sus ventajas y limitaciones. El objetivo de este análisis es comprender cómo se abordan actualmente estas necesidades en el sector y en qué medida las herramientas existentes inspiran o contrastan con el desarrollo del módulo Report Service en AESLA.

Microsoft Power BI¹ es una de las plataformas líderes en el ámbito de la inteligencia de negocio. Destaca por su capacidad de conexión a múltiples fuentes de datos y por ofrecer cuadros de mando interactivos y actualizados en tiempo real. Su principal fortaleza es la flexibilidad de visualización y la posibilidad de integrar gráficos avanzados sin conocimientos de programación. No obstante, entre sus limitaciones se encuentra la complejidad de configuración inicial y la necesidad de licencias específicas para acceder a todas las funcionalidades.

Tableau² constituye otra herramienta ampliamente utilizada. Su punto fuerte es la facilidad con la que permite generar visualizaciones intuitivas y estéticamente atractivas. Además, ofrece opciones de colaboración entre equipos y una buena integración con entornos corporativos. Sin embargo, su curva de aprendizaje puede ser elevada para usuarios sin experiencia previa, y su enfoque está más orientado a cuadros de mando interactivos que a informes exportables en formatos tradicionales como Excel.

Qlik Sense³ se centra en la exploración de datos y en la creación de dashboards personalizados. Su motor asociativo facilita el análisis de relaciones entre diferentes conjuntos

¹<https://www.microsoft.com/es-es/power-platform/products/power-bi>

²<https://www.tableau.com/es-es/why-tableau/what-is-tableau>

³<https://www.qlik.com/es-es/products/qlik-sense>

de datos, permitiendo descubrir patrones de forma ágil. Entre sus limitaciones destaca que, al igual que en el caso de Tableau, no siempre resulta la herramienta más adecuada para informes ejecutivos estandarizados en Excel, que siguen siendo demandados en muchos entornos corporativos.

JasperReports es una solución orientada a la generación programática de informes a partir de plantillas y fuentes de datos configurables [?]. Este enfoque se aproxima más al problema tratado en este TFG, aunque requiere desarrollar y mantener la lógica específica de integración y generación para cada dominio de negocio.

La comparación se resume en la tabla 2.1. Las valoraciones son cualitativas y se refieren a las características descritas en la documentación oficial de cada solución; no constituyen una prueba de rendimiento entre productos.

Cuadro 2.1: Comparación cualitativa de soluciones relacionadas con el análisis y la generación de informes.

Solución	Enfoque principal	Exportación a Excel	Generación de Excel personalizado	Integración con BIC
Power BI [?]	Inteligencia de negocio y visualización	Sí	No constituye su objetivo principal; permite exportar datos de visualizaciones	No específica
Tableau [?]	Análisis y visualización interactiva	Sí	Principalmente exportación de datos o vistas	No específica
Qlik Sense [?]	Analítica y exploración asociativa	Sí	Principalmente exportación de datos de visualizaciones	No específica
JasperReports [?]	Generación programática de informes	Sí	Sí, mediante plantillas y lógica de generación	Requiere integración específica
Report Service	Informes específicos sobre procesos BIC	Sí	Sí, adaptada a los requisitos de AESLA	Integración directa con la plataforma

Las soluciones analizadas responden a necesidades diferentes. Power BI, Tableau y Qlik Sense están orientadas principalmente al análisis visual y a los cuadros de mando, aunque permiten exportar los datos mostrados a formatos compatibles con Excel. JasperReports se aproxima más al problema tratado en este proyecto, al proporcionar un motor programático de generación de informes basado en plantillas.

El Report Service presenta, sin embargo, un ámbito más específico: se integra directamente con los datos y procesos de la plataforma BIC y genera libros Excel cuya estructura, hojas, métricas y reglas de negocio responden a los requisitos concretos de AESLA. Por tanto,

no pretende sustituir a una plataforma generalista de inteligencia de negocio, sino resolver una necesidad de reporting especializada dentro del ecosistema existente.

2.2 Tecnologías utilizadas

El desarrollo del Report Service se apoyó en un conjunto de tecnologías que abarcan tanto el cliente web como el servidor y la base de datos, junto con utilidades complementarias para despliegue, monitorización y calidad.

2.2.1 Servidor web

- **Spring Boot:** framework de desarrollo en Java que permite crear aplicaciones web y microservicios de forma rápida y estructurada. Aporta configuración automática, inyección de dependencias y un ecosistema rico de librerías [?].
- **Spring Batch:** módulo de Spring orientado al procesamiento por lotes. Se utilizó para implementar la generación asíncrona de informes, gestionando *jobs* y *steps* de manera controlada [?].
- **Hibernate / JPA:** framework de mapeo objeto-relacional que simplifica el acceso a la base de datos mediante entidades Java, evitando tener que escribir consultas [SQL](#) manuales [?].
- **Aspose Cells:** es una librería de generación y manipulación de Excel utilizada para dar formato, estilos y estructura a los informes [?].
- **Elasticsearch:** motor de búsqueda utilizado como fuente de datos adicional, especialmente para recuperar información de instancias de procesos con consultas [JSON](#) flexibles [?].
- **Maven:** herramienta de gestión y automatización de proyectos Java, empleada para manejar dependencias, construir el proyecto y ejecutar tareas comunes [?].

2.2.2 Cliente web

- **Angular:** framework de desarrollo basado en TypeScript utilizado para construir la interfaz del usuario. Permite crear componentes modulares, mantener un enrutamiento dinámico y gestionar formularios y servicios de forma escalable [?].

- **Yarn:** gestor de dependencias para proyectos JavaScript que garantiza instalaciones reproducibles y seguras. Fue la herramienta principal para administrar librerías del frontend y realizar auditorías de seguridad.
- **Node.js:** entorno de ejecución necesario para compilar y ejecutar [Angular](#) en local, así como para el servidor de desarrollo.
- **TypeScript:** lenguaje de programación que extiende JavaScript con tipado estático y características avanzadas, facilitando el desarrollo de aplicaciones robustas y mantenibles.

2.2.3 Base de datos

- **PostgreSQL:** sistema de gestión de bases de datos relacional, robusto y de código abierto, empleado para almacenar los informes generados y sus metadatos [?].
- **Flyway:** herramienta de migraciones que asegura el versionado y la coherencia del esquema de base de datos en todos los entornos (desarrollo, pruebas y producción). Se utilizó para crear y mantener el esquema *aesla_reports* y su tabla principal *report* [?].
- **Testcontainers:** framework de testing que permite levantar contenedores efímeros de [PostgreSQL](#) durante la ejecución de pruebas, garantizando un entorno controlado y reproducible [?].

2.2.4 Infraestructura y Despliegue

- **Docker:** plataforma de contenedores utilizada para empaquetar tanto el backend como el frontend y garantizar despliegues reproducibles en diferentes entornos [?].
- **Git:** sistema de control de versiones empleado en combinación con ramas de funcionalidad y *Pull Requests*, asegurando trazabilidad y revisiones de código antes de su integración.

2.2.5 Herramientas de apoyo

- **Gestión de trabajo:** [Jira](#) se utilizó junto con un tablero [Kanban](#) para organizar elementos de tipo *Epic*, historias de usuario y subtareas, así como para gestionar prioridades y realizar el seguimiento visual del progreso de cada ticket [?].
- **Control de versiones y hosting:** Git para control de versiones y GitHub como repositorio remoto centralizado donde se alojaba el código fuente del proyecto, facilitando la colaboración mediante *Pull Requests* y revisiones de código.

- **Flujo de trabajo Git:** [Gitflow](#) [?] se adoptó como convención de ramas (feature/fix/hotfix) para mantener estabilidad en las ramas de producción y organizar el desarrollo de forma estructurada.
- **Integración continua / CI & CD:** [GitHub Actions](#) se configuró con acciones personalizadas para automatizar el build del proyecto, ejecutar pruebas unitarias e integradas, y realizar despliegues [?]. Los pipelines se integraron con SonarQube (SonarCloud) para validar métricas de calidad de código ([Coverage](#), [Duplicación de código](#), [Code Smells](#)) antes de aceptar cada [PR](#) [?].
- **Registro y gestión de artefactos:** [Quay](#) se empleó como registry principal para almacenar y versionar las imágenes Docker de los entornos Ansible de prueba, mientras que [JFrog Artifactory](#) sirvió como repositorio de artefactos binarios y dependencias Maven/NPM, facilitando la distribución controlada de versiones.
- **Orquestación de despliegue y automatización:** [Ansible](#) (gestionado mediante [AWX](#)) permitió levantar entornos de prueba y automatizar despliegues en [QA](#).
- **Entornos de desarrollo:** IntelliJ IDEA para backend (Java/Spring) y Visual Studio Code para frontend (Angular/TypeScript).
- **Pruebas y depuración:** [Postman](#) para pruebas de [API](#) y Google Chrome para pruebas manuales en entorno local.
- **Calidad de código:** [SonarCloud](#) / SonarQube se integró en los pipelines de [CI](#) para realizar análisis estático de código ([HTML](#), [CSS](#), TypeScript, Java), medir cobertura de pruebas, detectar duplicaciones y controlar la deuda técnica, garantizando que cada cambio cumpliera con los umbrales de calidad definidos por la empresa.

Metodología y planificación

3.1 Metodología de desarrollo

El proyecto se llevó a cabo siguiendo una metodología ágil [?] basada en iteraciones cortas y gestión Kanban [?] en [Jira](#). Este enfoque resultó adecuado por la naturaleza incremental de las mejoras del Report Service, la necesidad de integrar cambios en backend y frontend con validación continua, y la conveniencia de priorizar incidencias, tareas técnicas y funcionalidades nuevas. Este tablero Kanban se configuró con el objetivo de permitir visualizar el flujo de trabajo completo, con estados definidos (*To Do* → *Task Breakdown* → *In Progress* → *Ready for QA/PR* → *QA/PR OK* → *Done*) para cada tarea, facilitando el seguimiento de prioridades y la identificación de bloqueos.

Las características principales del enfoque fueron:

- **Iterativo e incremental:** cada conjunto de cambios (feature/refactor/fix/chore) se gestionó como un elemento de trabajo en el tablero, produciendo un incremento funcional verificable (código, tests y PR).
- **Retroalimentación continua:** reuniones diarias con el equipo para sincronizar prioridades, desbloquear tareas y validar entregables; revisión de PRs con comentarios técnicos.
- **Flexibilidad:** replanificación rápida en función de prioridades del [EPIC](#) y de incidencias no previstas inicialmente que surgieron durante el desarrollo (p. ej., unificación de estructuras de peticiones entre tipos de informe, compatibilidad de parámetros opcionales como fechas nulas, detección de duplicidades), las cuales se integraron de manera ágil en las tareas correspondientes.

3.1.1 Adaptación al proyecto

La incorporación al proyecto siguió una fase breve de formación, alineada con las tecnologías clave:

- **Formación inicial:** cursos introductorios de Spring Batch, [Docker](#), patrones de diseño en Java y Angular (priorizando primero el backend), proporcionados mediante la suscripción de Udemy contratada por la empresa.
- **Aterrizaje en Jira:** asignación al [EPIC Improvements for AESLA Project in 2025 Q3 & Q4](#), que agrupaba los work items relacionados con la evolución del Report Service. La mayoría se clasificaron como *Task*, aunque también se gestionaron incidencias de tipo *Bug* y una historia de usuario de mayor alcance para introducir la generación asíncrona.
- **Flujo de trabajo:**
 - Rama por tarea (feature/fix branch) en Git.
 - Commits atómicos y descriptivos.
 - Pull Request hacia *development*, con code review y checks automáticos de SonarCloud y build de pull/push.
 - Cierre de la tarea al completar [PR](#) y [QA](#).
 - Uso de una misma *fix version* del Report Service para agrupar los cambios que debían publicarse conjuntamente.

El orden de adaptación técnica fue: backend → datos → frontend, permitiendo una integración progresiva. Para cada work item se creó una rama independiente en Git, identificada con el número de la tarea, donde se realizaron los commits correspondientes. Una vez finalizado el desarrollo y superada la revisión de código y la validación de QA, se aprobaba el Pull Request y se integraban los cambios en la rama *development* del Report Service.

3.2 Planificación y seguimiento

El proyecto se desarrolló durante los meses de julio, agosto y septiembre. La dedicación al trabajo fue de 7 horas diarias durante julio y agosto, y de 6 horas diarias durante septiembre, al tener que compaginar las prácticas con el inicio del curso universitario.

La planificación del proyecto se estructuró en fases de trabajo, en lugar de *sprints* formales, de acuerdo con la metodología ágil basada en Kanban utilizada por el equipo. Esta forma de organización permitió adaptar el ritmo de trabajo y la prioridad de las tareas a las necesidades reales del proyecto, manteniendo un seguimiento continuo del estado de cada *work item*.

Con el objetivo de reflejar tanto la previsión inicial como la evolución efectiva del proyecto, se presentan dos diagramas de Gantt: uno correspondiente a la **planificación inicial** y otro al **seguimiento real**. La comparación entre ambos permite identificar de forma visual las fases que se prolongaron más de lo previsto inicialmente.

3.2.1 Planificación inicial

La planificación inicial se definió de forma orientativa al comienzo del proyecto. En ella se preveía una primera fase breve de incorporación y formación, seguida por tareas de refactorización y limpieza del código. Posteriormente se abordaría la unificación de la gestión de informes y la evolución del backend como núcleo principal del trabajo. Finalmente, la integración del frontend se reservaría para septiembre, dejando la última semana para las tareas de cierre, liberación de versión y documentación.

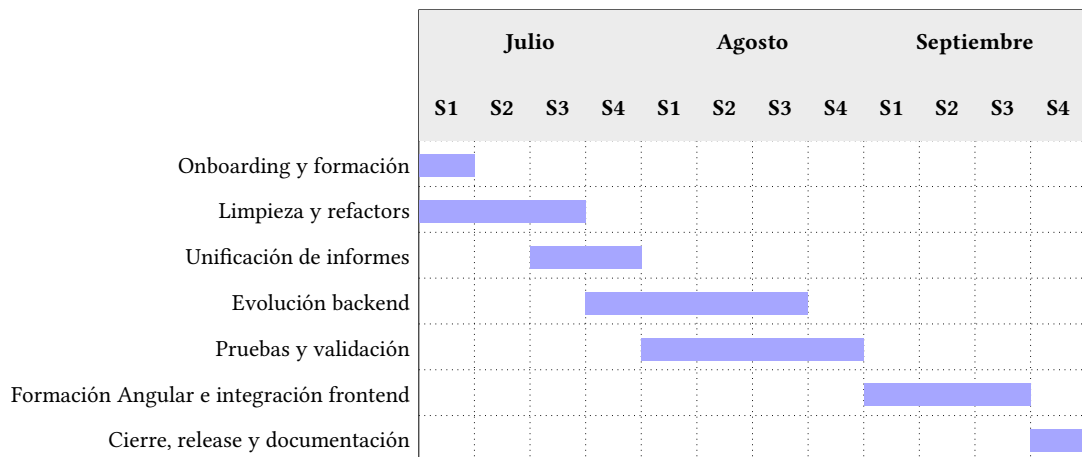


Figura 3.1: Diagrama de Gantt de la planificación inicial del proyecto.

Como puede observarse, la planificación inicial planteaba una secuencia relativamente ordenada: una primera fase de adaptación durante la primera semana de julio, una etapa de refactorización durante el resto del mes, la transición hacia la unificación del servicio entre finales de julio y comienzos de agosto, el desarrollo del backend como fase central del proyecto, y finalmente la integración del frontend y el cierre durante septiembre.

3.2.2 Seguimiento real

Durante la ejecución del proyecto, la planificación inicial tuvo que adaptarse a las necesidades que fueron surgiendo, en coherencia con la metodología ágil empleada. Algunas tareas previstas resultaron más complejas de lo estimado inicialmente, especialmente aquellas relacionadas con la unificación de la gestión de informes y la evolución del backend.

La unificación de la gestión de informes se prolongó más de lo previsto porque implicó homogeneizar controladores, endpoints y DTOs, así como normalizar el tratamiento de parámetros opcionales y reorganizar la estructura interna de paquetes. Del mismo modo, la evolución del backend también requirió más tiempo, al incorporar persistencia de informes, endpoints de consulta y descarga, generación asíncrona con Spring Batch, control de duplicidad, registro de métricas y ajustes relacionados con la inicialización de Flyway y de las tablas asociadas a Spring Batch.

Además, las pruebas y validaciones no se limitaron a una fase puntual, sino que se mantuvieron a lo largo de buena parte del desarrollo, en forma de pruebas unitarias, integradas, revisión de *Pull Requests* y validación en QA. Como consecuencia, varias actividades se solaparon parcialmente sin alterar la fecha final de cierre del proyecto.

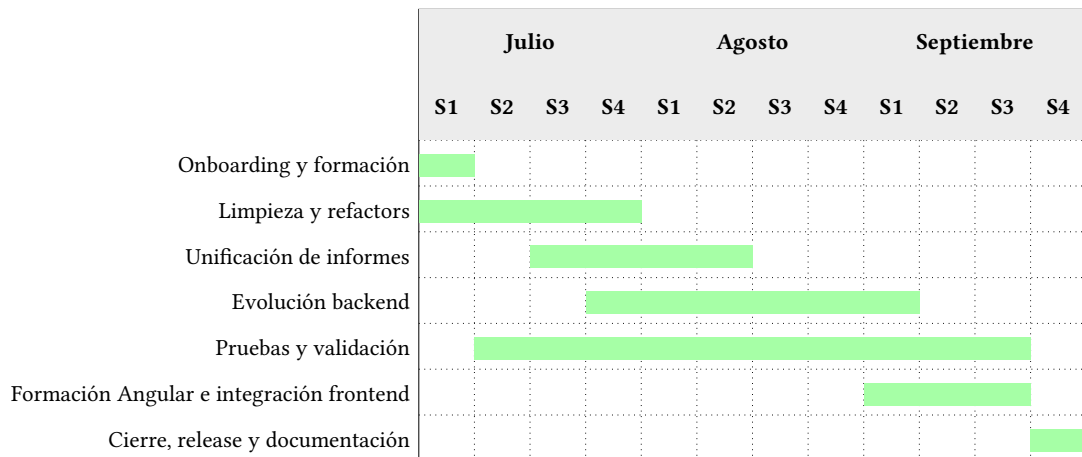


Figura 3.2: Diagrama de Gantt correspondiente al seguimiento real del proyecto.

La comparación entre ambos diagramas muestra que las principales desviaciones se produjeron en la fase de unificación de la gestión de informes y en la fase de evolución backend, ambas con una duración superior a la prevista inicialmente. Asimismo, las pruebas y validación adquirieron un carácter transversal, acompañando al desarrollo durante gran parte del proyecto. No obstante, estas desviaciones no afectaron al cumplimiento de los objetivos ni al cierre del proyecto dentro del periodo inicialmente establecido.

4.1 Requisitos

4.1.1 Actores y Sistemas involucrados

En el sistema intervienen distintos actores y componentes que participan en la generación, almacenamiento y consulta de los informes ejecutivos:

- **Usuario AESLA:** empleado o analista encargado de generar y consultar los informes a través de la interfaz web. Puede lanzar nuevas generaciones, visualizar el historial y volver a descargar informes anteriores.
- **Aplicación web:** cliente desarrollado en Angular que permite la interacción del usuario con el sistema. Presenta formularios, mensajes de estado y botones de acción, y consulta periódicamente el estado de la generación mediante el endpoint `/api/reports/latest`.
- **Servicio de reportes:** aplicación de servidor desarrollada con Spring Boot. Gestiona la lógica de negocio, coordina la generación asíncrona de informes, controla la duplicidad y persiste los resultados.
- **Origen de datos de procesos:** base de datos de ejecución de procesos de BIC (BIC Process Execution) y fuentes adicionales en *Elasticsearch*, de donde se obtienen las variables necesarias para la composición de los informes.
- **Base de datos de informes:** sistema PostgreSQL donde se almacenan los informes generados, junto con sus metadatos (categoría, tipo, fechas, franquicias, tamaño, etc.).

4.1.2 Requisitos funcionales

Los requisitos funcionales definen el comportamiento observable del sistema y las acciones que el usuario puede realizar:

1. RF-01 - Generación de informes:

- El sistema debe permitir la generación de informes en formato Excel, ofreciendo la posibilidad de seleccionar los siguientes parámetros: *tipo* (KPI o Producto), *categoría* (Innovación o Alternativa), *franquicia(s)* (una, varias o todas) y *rango de fechas* (opcional), que por defecto abarcará desde el inicio de la disponibilidad de datos.
- La generación se debe realizar de forma asíncrona, sin bloquear la interfaz.
- La aplicación debe mostrar mensajes informativos durante la generación y su seguimiento mediante *polling*: “Comprobando solicitud”, “Generando informe”, “Descarga iniciada”, “Error de red” o “Informe reciente disponible en la pestaña Historial”.

2. RF-02 - Control de duplicidad:

- Antes de generar un nuevo informe, el sistema debe comprobar si existe un informe idéntico generado recientemente (p. ej., en los últimos 5 minutos).
- En caso afirmativo, se debe bloquear la nueva generación y notificar al usuario que el informe ya está disponible en el historial.
- Para realizar esta comprobación, se genera una firma única basada en los parámetros de entrada.

3. RF-03 - Descarga y redescarga de informes:

- Al finalizar una generación válida, el informe debe quedar disponible para su descarga desde la interfaz del usuario.
- El informe debe almacenarse en la base de datos para su consulta futura.
- El usuario podrá volver a descargar cualquier informe desde la pestaña *Historial*.

4. RF-04 - Historial de informes:

- El sistema debe disponer de una página de *Historial* que muestre todos los informes generados, junto con sus metadatos: fecha de creación, categoría de proceso, tipo de informe, franquicias a las que hace referencia (marcas) y rango de fechas.
- El historial debe estar ordenado de forma descendente (más recientes primero).
- Debe implementarse paginación mediante botones “5 anteriores” y “5 siguientes”.

4.1.3 Requisitos no funcionales

Los requisitos no funcionales garantizan la calidad y las propiedades del sistema:

- **RNF-01 - Usabilidad:** interfaz simple e intuitiva, con botones y mensajes claros que guíen al usuario en todo momento.
- **RNF-02 - Rendimiento:** la ejecución de informes no debe bloquear la aplicación; la respuesta del historial debe ser inmediata.
- **RNF-03 - Confiabilidad:** almacenamiento persistente en base de datos; control de duplicidad para evitar sobrecarga.
- **RNF-04 - Mantenibilidad:** estructura modular en capas (controladores, servicios, utilidades) y cobertura de pruebas unitarias e integradas.
- **RNF-05 - Escalabilidad:** la arquitectura debe permitir la ejecución asíncrona de trabajos y facilitar una futura ampliación de la capacidad de procesamiento sin requerir cambios sustanciales en el flujo funcional.
- **RNF-06 - Seguridad:** integración con los mecanismos de autenticación y autorización proporcionados por la plataforma BIC, junto con la protección del contenido de los informes.
- **RNF-07 - Observabilidad:** registro de métricas, logs estructurados y seguimiento de procesos.
- **RNF-08 - Compatibilidad:** funcionamiento correcto en navegadores modernos y exportación a formatos Excel estándar.

4.1.4 Historias de usuario

A continuación se detallan las principales historias de usuario definidas para el proyecto:

- **HU01 – Generar informe:** Como usuario, quiero seleccionar los parámetros de tipo, categoría, franquicias y rango de fechas para generar un informe Excel personalizado.
- **HU02 – Visualizar estados de generación:** Como usuario, quiero recibir notificaciones sobre el estado del informe (comprobando, generando, completado, error, duplicado) para conocer el progreso.
- **HU03 – Control de duplicidad:** Como usuario, quiero que el sistema detecte y bloquee la generación de informes idénticos para evitar duplicidades innecesarias.
- **HU04 – Descarga del informe:** Como usuario, quiero descargar el informe una vez que la generación haya finalizado.

- **HU05 – Historial de informes:** Como usuario, quiero poder consultar un listado con todos los informes generados, ver su información.
- **HU06 – Redescarga:** Como usuario, quiero volver a descargar un informe anterior sin tener que generarlo de nuevo.

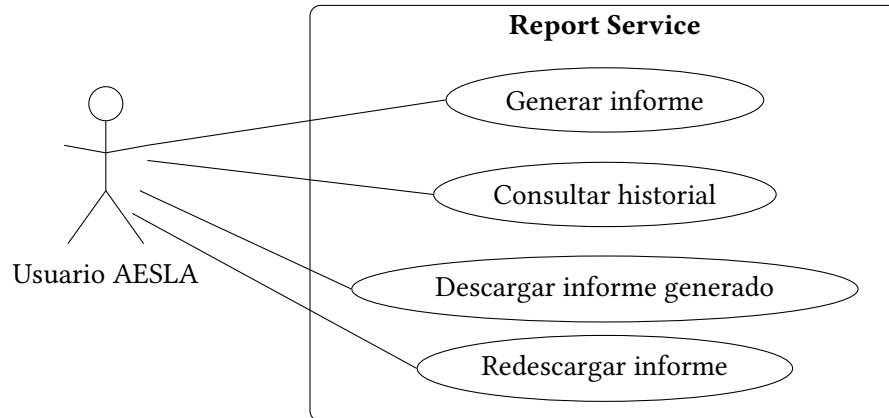


Figura 4.1: Diagrama de casos de uso principales del Report Service. Fuente: elaboración propia.

La figura 4.1 resume las principales funcionalidades accesibles para el usuario. El seguimiento del estado mediante *polling*, el control de duplicidad y la comprobación de disponibilidad de datos forman parte del comportamiento interno asociado al caso de uso de generación de informes.

4.2 Arquitectura del sistema

La aplicación sigue una arquitectura cliente-servidor basada en servicios web [REST](#) y procesamiento asíncrono. En este capítulo se presenta una visión funcional de los actores y responsabilidades principales; la descripción técnica detallada de paquetes, clases y flujo de ejecución se desarrolla en el capítulo 5.

4.2.1 Cliente web

El cliente web, desarrollado en Angular, gestiona la interacción del usuario y las peticiones al servidor:

- Página principal: módulo *Report Generator*, donde se configuran los parámetros y se lanza la generación.

- Página secundaria: módulo *Report History*, donde se visualizan y descargan los informes existentes.
- Comunicación con el servidor mediante servicios [HTTP](#), con notificaciones de estado y validaciones de entrada.

4.2.2 Servidor

El servidor gestiona la lógica principal:

- Controladores REST que reciben solicitudes de generación.
- Servicios de coordinación que controlan la duplicidad y comprueban la disponibilidad de datos antes y durante la ejecución del job.
- Ejecución asíncrona mediante Spring Batch, con jobs y steps configurables.
- Persistencia de resultados y metadatos en PostgreSQL.
- Generación de ficheros Excel con Aspose Cells.

4.2.3 Capa de datos

- Fuentes de información: BIC Process Execution y Elasticsearch, de las que se extraen las variables necesarias.
- Almacenamiento: PostgreSQL con migraciones controladas por Flyway.

El flujo funcional comienza en el cliente Angular, que envía la solicitud al controlador REST. El servidor valida los parámetros, comprueba posibles duplicidades y, si la solicitud es válida, lanza un job de Spring Batch. El tasklet comprueba la disponibilidad de datos, selecciona el generador KPI o Producto correspondiente, obtiene y transforma los datos, crea el workbook y lo almacena en PostgreSQL. Mientras el job se ejecuta, el frontend consulta periódicamente el endpoint `/api/reports/latest` con los parámetros de la solicitud. Cuando la respuesta indica que el informe está disponible, el cliente inicia automáticamente la descarga; si no, continúa consultando hasta alcanzar el tiempo máximo configurado.

4.3 Interfaz de usuario

La interfaz de usuario original del Report Service presentaba un diseño sencillo y funcional, pero con limitaciones evidentes desde el punto de vista de la experiencia de usuario. Se

trataba de una única página web donde el usuario podía configurar los parámetros básicos del informe: tipo (KPI o Producto), categoría (Innovación o Alternativa), selección de franquicias y un rango de fechas opcional.

Una vez configurados estos parámetros, el usuario podía solicitar la generación del informe mediante un botón de descarga. En la versión original, el sistema procesaba la solicitud de forma síncrona, bloqueando la interfaz durante todo el proceso de generación, lo que podía resultar en tiempos de espera prolongados sin retroalimentación clara sobre el estado del proceso. Al finalizar, el informe Excel se descargaba directamente al navegador del usuario.

El diseño inicial, aunque funcional, presentaba carencias de usabilidad. Las principales limitaciones eran: ausencia de historial, falta de retroalimentación durante la generación, procesamiento síncrono que bloqueaba la interfaz, diseño anticuado y ausencia de control de duplicidad. Estas carencias motivaron el desarrollo de una nueva interfaz modernizada (capítulo 7).

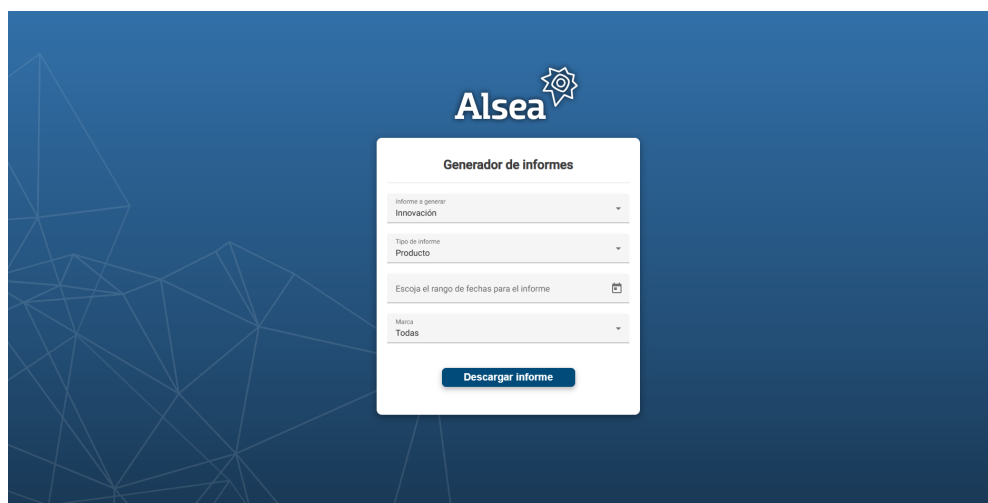


Figura 4.2: Interfaz original del Report Service antes de su modernización.

5.1 Arquitectura tecnológica del sistema

En este apartado se describe la estructura final del Report Service modernizado, justificando las principales decisiones técnicas y explicando cómo interactúan los diferentes componentes que participan en la generación, almacenamiento, consulta y descarga de informes.

5.1.1 Visión general de la arquitectura

El Report Service sigue una arquitectura cliente-servidor organizada en tres capas principales —presentación, lógica de aplicación y datos—, complementadas por los componentes de infraestructura necesarios para la migración de base de datos, contenerización y despliegue. Esta separación permite distribuir las responsabilidades del sistema y facilita su mantenimiento y evolución.

- **Capa de presentación:** aplicación web desarrollada en Angular 15 que proporciona la interfaz necesaria para configurar la generación de informes, consultar su estado, acceder al historial y descargar informes previamente generados.
- **Capa de lógica de aplicación:** backend basado en Spring Boot 3 y Spring Batch encargado de recibir las peticiones, realizar las validaciones necesarias, comprobar la duplicidad, coordinar la generación asíncrona y gestionar el almacenamiento de los resultados.
- **Capa de datos:** PostgreSQL almacena los informes generados y sus metadatos, mientras que Elasticsearch y BIC Process Execution actúan como fuentes de información para la construcción de los diferentes tipos de informe.

La tabla [5.1](#) resume los componentes más relevantes del sistema y sus responsabilidades.

Cuadro 5.1: Componentes principales y responsabilidades del Report Service.

Componente	Responsabilidad principal
Cliente Angular	Recoger los parámetros, validar la entrada, mostrar el estado de la solicitud y permitir consultar y descargar informes.
<i>ReportBatchController</i>	Exponer los endpoints REST necesarios para iniciar generaciones, consultar el estado y el historial y solicitar descargas.
<i>ReportLaunchService</i>	Coordinar la comprobación previa de duplicidad y el lanzamiento asíncrono del job.
<i>ReportDuplicateUtils</i>	Calcular la firma de la solicitud y facilitar la detección de solicitudes equivalentes desencadenadas recientemente.
<i>GenerateReportTasklet</i>	Coordinar la ejecución del job, comprobar la disponibilidad de datos, seleccionar el generador correspondiente y delegar el almacenamiento del resultado.
<i>ReportDataProbeService</i>	Comprobar si existen datos suficientes para los parámetros solicitados antes de ejecutar la generación completa del informe.
<i>ReportService</i>	Gestionar la serialización, persistencia y recuperación de los informes generados.
<i>ReportRepository</i>	Proporcionar mediante JPA el acceso a los informes almacenados en PostgreSQL.

La figura 5.1 muestra de forma resumida el flujo principal de generación y las dependencias entre los componentes más importantes.

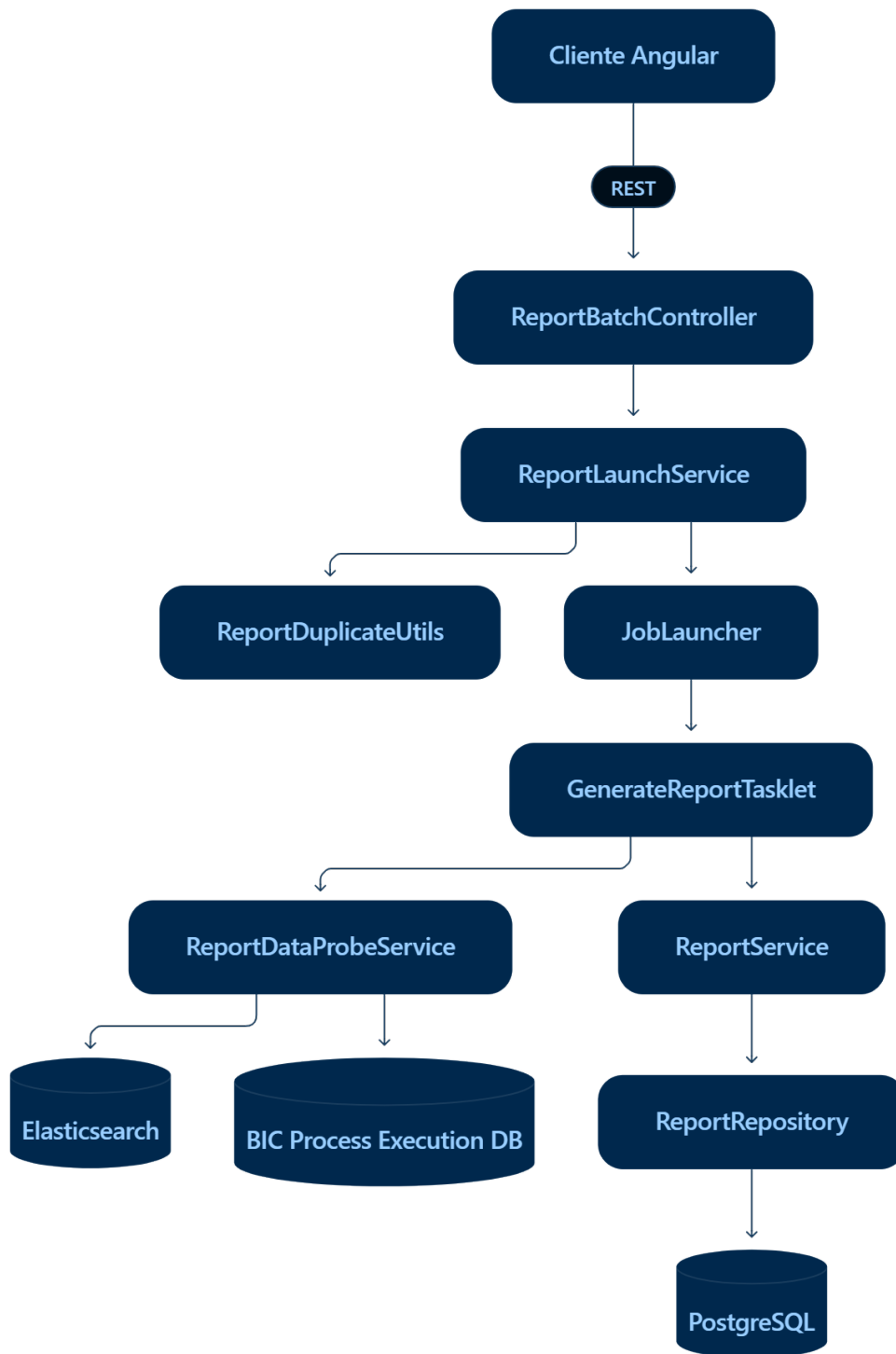


Figura 5.1: Arquitectura simplificada del flujo principal de generación de informes. Fuente: elaboración propia.

El cliente Angular envía las solicitudes al backend mediante la API REST expuesta por *ReportBatchController*, que delega la coordinación de una nueva generación en *ReportLaunchService*.

Antes de iniciar el procesamiento, *ReportLaunchService* comprueba mediante *ReportDuplicateUtils* y *ReportRepository* si una solicitud equivalente ha sido desencadenada recientemente. Si no existe duplicidad, se inicia de forma asíncrona un job de Spring Batch mediante *JobLauncher*.

Durante la ejecución del job, *GenerateReportTasklet* comprueba mediante *ReportDataProbeService* que existen datos para los parámetros solicitados y selecciona el generador correspondiente según el tipo y la categoría del informe. Una vez construido el workbook, el tasklet delega su almacenamiento en *ReportService*, que realiza la serialización del fichero y utiliza *ReportRepository* para persistirlo en PostgreSQL.

La figura del cuerpo principal pretende ofrecer una visión sintética del flujo de generación. Una representación más completa, que incorpora los componentes internos del frontend y del backend, las fuentes de datos, Spring Batch y los elementos de infraestructura y despliegue, se incluye en el apéndice A.1.

5.1.2 Comunicación entre componentes

La comunicación entre el cliente y el servidor se realiza mediante una [API REST](#) sobre [HTTP](#), utilizando [JSON](#) para el intercambio de información estructurada y contenido binario para la descarga de los ficheros Excel.

Los principales endpoints utilizados son:

- *POST /api/reports*: recibe los parámetros de una nueva generación. Antes de iniciar el procesamiento, *ReportLaunchService* comprueba si se ha desencadenado recientemente una solicitud equivalente. Si no existe duplicidad, el job se lanza de forma asíncrona y la petición queda aceptada para su procesamiento.
- *GET /api/reports/latest*: permite consultar periódicamente si el informe correspondiente a una solicitud ya se encuentra disponible. Este endpoint constituye la base del mecanismo de *polling* utilizado por el frontend.
- *GET /api/reports/history*: devuelve el listado paginado de informes almacenados, ordenados de más reciente a más antiguo.
- *GET /api/reports/{id}/download*: recupera el contenido binario de un informe almacenado a partir de su identificador [UUID](#).

La comprobación de disponibilidad de datos no se realiza antes de lanzar el job, sino durante su ejecución. Una vez iniciado, *GenerateReportTasklet* utiliza *ReportDataProbeService.hasData()* para determinar si existen datos para la combinación solicitada. Si no existen, el step finaliza con el estado *NO_DATA* sin construir el fichero Excel.

5.1.3 Justificación de la arquitectura elegida

La arquitectura adoptada responde principalmente a criterios de separación de responsabilidades, mantenibilidad y desacoplamiento del procesamiento de larga duración.

- **Modularidad:** cada componente concentra una responsabilidad específica. El cliente gestiona la interacción con el usuario; los controladores exponen la API; los servicios coordinan la lógica de aplicación; y los repositorios encapsulan el acceso a datos.
- **Separación de responsabilidades:** el controlador no contiene la lógica específica de construcción de informes. Esta responsabilidad se delega en Spring Batch, el tasklet y los servicios especializados de cada tipo y categoría de informe.
- **Procesamiento asíncrono:** el uso de Spring Batch y de un executor asíncrono permite desacoplar la generación del ciclo principal de la petición HTTP, evitando mantener bloqueada la interfaz mientras se construye el informe.
- **Persistencia independiente:** la centralización del almacenamiento en *ReportService* y *ReportRepository* permite separar la lógica de generación de la forma en que los informes son almacenados y recuperados.
- **Trazabilidad:** Flyway mantiene versionados los cambios en el esquema de base de datos, mientras que las tablas de metadatos de Spring Batch permiten conservar información sobre las ejecuciones de los jobs.
- **Capacidad de evolución:** la división entre tipos de informe, categorías y servicios comunes facilita la incorporación de nuevas variantes sin modificar de forma sustancial los puntos de entrada de la aplicación.

No se realizaron pruebas formales de carga dentro del alcance del proyecto, por lo que no se pretende cuantificar el comportamiento del sistema ante niveles elevados de concurrencia. La arquitectura sí proporciona una base desacoplada sobre la que podrían estudiarse futuras mejoras de capacidad y escalabilidad.

5.1.4 Stack tecnológico

El stack tecnológico final utilizado se compone de:

- **Cliente:** Angular 15, TypeScript, Angular Material, RxJS y Yarn.
- **Servidor:** Spring Boot 3, Spring Batch, Hibernate/JPA y Maven.
- **Almacenamiento:** PostgreSQL para los informes y sus metadatos, y Elasticsearch como una de las fuentes de información de los procesos.
- **Generación de Excel:** Aspose Cells para la construcción y el formato de los workbooks.
- **Migraciones:** Flyway para el control de versiones del esquema de base de datos.
- **Infraestructura:** Docker para contenerización y Ansible/AWX para automatización del despliegue.

5.2 Diseño de la aplicación

A continuación se detalla el diseño interno de los principales componentes del sistema, describiendo la organización del cliente y del servidor, el flujo de generación y la capa de persistencia.

5.2.1 Cliente

El cliente web se organiza mediante componentes y servicios Angular que separan la presentación de la comunicación con el backend.

Estructura de módulos y componentes

El frontend del proyecto está ubicado en *webapp/custom-aesla-angular-front*. La organización relevante dentro de *src/app* es la siguiente:

- **reports-generator:** contiene el componente principal de generación de informes.
 - *reports-generator.component.ts*: lógica del componente.
 - *reports-generator.component.html*: plantilla del formulario.
 - *reports-generator.component.scss*: estilos específicos.
 - *reports-generator.component.spec.ts*: pruebas unitarias.
- **reports-history:** agrupa el componente encargado del historial y de su tabla paginada.
 - *reports-history.component.ts*: lógica del componente.

- *reports-history.component.html*: plantilla de la tabla.
 - *reports-history.component.scss*: estilos del historial.
 - *reports-history.component.spec.ts*: pruebas unitarias.
 - *dialog/*: diálogos auxiliares utilizados por la interfaz.
- **services**: contiene los servicios responsables de la comunicación HTTP con el backend.
 - *get-alternative-franchise-list.service.ts*.
 - *get-innovation-franchise-list.service.ts*.
 - *report-download.service.ts*.
 - *reports-history.service.ts*.
 - **Archivos raíz**:
 - *app.component.ts*: componente raíz.
 - *app.module.ts*: módulo principal.
 - *app-routing.module.ts*: configuración de las rutas entre generación e historial.

Esta organización mantiene separadas las principales responsabilidades del cliente: el componente de generación gestiona la creación de informes, el historial permite consultar y redescargar los existentes y los servicios centralizan las comunicaciones con el backend.

Servicios Angular

Los servicios encapsulan las peticiones HTTP y la gestión de las respuestas recibidas desde el backend.

- ***get-alternative-franchise-list.service.ts***: obtiene las franquicias disponibles para informes de categoría Alternativa.
- ***get-innovation-franchise-list.service.ts***: realiza la misma función para la categoría Innovación.
- ***report-download.service.ts***: gestiona el flujo de generación y descarga. Entre sus responsabilidades se encuentran:
 - enviar la petición *POST /api/reports*;
 - realizar el seguimiento de la generación mediante *polling*;
 - solicitar la descarga una vez disponible el informe;

- recibir el fichero Excel como contenido binario;
 - gestionar los mensajes de estado y situaciones de error.
- **reports-history.service.ts**: centraliza las operaciones relacionadas con el historial.
 - *getHistory(page, size)* obtiene el listado paginado mediante *GET /api/reports/history*.
 - *downloadReport(id)* recupera un informe almacenado mediante *GET /api/reports/{id}/download*.
 - transforma las respuestas recibidas a los modelos TypeScript utilizados por los componentes.

Los servicios utilizan *HttpClient* para las peticiones HTTP y Observables de RxJS para la gestión de los flujos asíncronos. Asimismo, utilizan el mecanismo de inyección de dependencias de Angular mediante *@Injectable*.

Flujo de datos entre componentes

El flujo habitual de generación de un informe puede resumirse en los siguientes pasos:

1. El usuario configura en *ReportsGeneratorComponent* el tipo, la categoría, las franquicias y, cuando procede, el rango de fechas.
2. Al seleccionar una categoría se consulta dinámicamente la lista de franquicias mediante el servicio correspondiente.
3. Al pulsar el botón de generación, el componente valida los parámetros introducidos.
4. Si los parámetros son válidos, *ReportDownloadService* envía una petición *POST /api/reports*.
5. El backend comprueba mediante *ReportDuplicateUtils* y el repositorio si se ha desencadenado una solicitud equivalente dentro de la ventana temporal definida.
6. Si se detecta una duplicidad, no se inicia un nuevo job y se informa al usuario de la existencia de una solicitud reciente.
7. Si la solicitud puede procesarse, *ReportLaunchService* inicia de forma asíncrona el job de Spring Batch y el cliente comienza el seguimiento del resultado.
8. Durante la ejecución del job, *GenerateReportTasklet* utiliza *ReportDataProbeService* para comprobar si existen datos. Si no existen, el procesamiento finaliza con el estado *NO_DATA*.

9. Mientras la generación continúa, *ReportDownloadService* consulta periódicamente */api/reports/latest*. En la implementación utilizada, la consulta se realiza cada cinco segundos durante un máximo de 120 segundos.
10. Cuando el informe se encuentra disponible, la respuesta incluye su identificador y el cliente solicita */api/reports/{id}/download*, recibe el contenido como *blob* y comienza la descarga.
11. Independientemente de la descarga inmediata, el informe queda almacenado y puede recuperarse posteriormente desde *ReportsHistoryComponent*.

Validaciones y feedback al usuario

El formulario de generación aplica validaciones antes de enviar la solicitud:

- tipo y categoría de informe obligatorios;
- selección de al menos una franquicia;
- comprobación de coherencia del rango de fechas cuando este se especifica;
- presentación de mensajes asociados a errores de validación.

Durante el proceso, la interfaz muestra mensajes relacionados con las distintas situaciones del flujo, como la comprobación de la solicitud, el inicio de la generación, la disponibilidad del fichero, la existencia de un informe reciente o los errores producidos.

El seguimiento no se realiza mediante una conexión persistente entre cliente y servidor. El frontend consulta periódicamente */api/reports/latest*, por lo que el mecanismo utilizado corresponde a *polling* y no a una comunicación mediante WebSocket o Server-Sent Events.

Frameworks y librerías de interfaz

El cliente utiliza principalmente:

- **Angular Material**: biblioteca de componentes de interfaz que proporciona formularios, tablas, botones, diálogos y *snackbars* [?].
- **RxJS**: biblioteca empleada para trabajar con flujos asíncronos mediante Observables [?].
- **Yarn**: gestor utilizado para la instalación y resolución de las dependencias del frontend.

5.2.2 Servidor (Spring Boot + Spring Batch)

El backend sigue una organización modular en capas, utilizando el mecanismo de inyección de dependencias proporcionado por Spring. El código separa la configuración, los puntos de entrada REST, la coordinación del procesamiento, la lógica específica de cada tipo de informe y la persistencia.

Estructura de paquetes

El backend está ubicado en *main/java/com/gbtec/bic/cloud*. Los paquetes más relevantes para el Report Service son:

- **config**: configuración general de Spring.
 - *AsposeConfiguration*: configuración de Aspose Cells.
 - *WebSecurityConfig*: configuración de seguridad web y CORS.
- **datastorage.elasticsearch**: integración con Elasticsearch.
 - *ElasticsearchService*: servicio utilizado para consultar variables de procesos.
- **processes.batch**: infraestructura relacionada con Spring Batch.
 - *BatchAsyncConfiguration*: configuración del executor asíncrono.
 - *ReportBatchConfig*: definición del job y de su step.
 - *FranchiseController*: endpoints relacionados con franquicias.
 - *ReportBatchController*: endpoints de generación y consulta de informes.
 - *ReportParamService*: preparación de parámetros del job.
 - *ReportRequestDto*: representación de la solicitud.
 - *GenerateReportTasklet*: ejecución de la generación.
 - *ReportDuplicateUtils*: cálculo de la firma y comprobación de duplicidad.
 - *ReportFileNameUtils*: construcción de nombres de fichero.
 - *ReportParamUtils*: normalización y tratamiento de parámetros.
- **kpi_reports**: lógica específica para informes KPI, organizada por las categorías *alternative* e *innovation*. Cada categoría contiene sus modelos, repositorios, servicios y utilidades correspondientes.

- **product_reports**: lógica específica para los informes de Producto, también separada entre *alternative* e *innovation*, con componentes de acceso a datos y generación.
- **report_probe**: funcionalidad de comprobación de disponibilidad de datos y coordinación del lanzamiento.
 - *ReportDataProbeService*.
 - *ReportDataProbeServiceImpl*.
 - *ReportLaunchService*.
- **report_storage**: persistencia de los informes.
 - *ReportEntity*.
 - *ReportRepository*.
 - *ReportService*.
- **utils**: utilidades compartidas relacionadas principalmente con la construcción y el formato de los libros Excel.
 - *heatmap/*.
 - *CreateExcelFileUtils*.
 - *ExcelCellStyleUtils*.
- **ui**: configuración y controladores utilizados para servir la interfaz.

Esta organización separa la configuración del procesamiento batch, la lógica específica de los informes, la comprobación de datos, la persistencia y las utilidades comunes.

En particular, *BatchAsyncConfiguration* configura el executor asíncrono y *ReportBatchConfig* define el job y el step de generación. *ReportBatchController* actúa como punto de entrada REST, mientras que *ReportRequestDto* agrupa los parámetros necesarios para identificar el tipo de informe, la categoría, las franquicias y las fechas opcionales.

Patrones y principios de diseño

El backend aplica varios mecanismos habituales del ecosistema Spring:

- **Arquitectura en capas**: separación entre controladores, servicios y repositorios.
- **Inyección de dependencias**: Spring gestiona las dependencias entre componentes y su ciclo de vida.

- **DTO:** los parámetros de la generación se agrupan en *ReportRequestDto*, evitando acoplar la representación externa de las solicitudes a las entidades de persistencia.
- **Delegación de responsabilidades:** el controlador no genera directamente los informes, sino que delega la coordinación en los servicios y la ejecución en Spring Batch.

Organización de los generadores de informes

La lógica de generación se divide en dos ramas principales: *kpi_reports* y *product_reports*. Cada una se divide a su vez entre las categorías *alternative* e *innovation*.

Esta separación es necesaria porque los informes KPI y Producto trabajan con modelos de datos y métricas diferentes.

En los informes KPI, los servicios correspondientes recuperan las instancias y los proyectos necesarios, calculan indicadores y generan las hojas específicas del workbook.

En los informes de Producto, los componentes de acceso a datos recuperan la información necesaria para construir elementos como retroplannings, informes diarios y heatmaps.

Las operaciones compartidas relacionadas con estilos, creación de celdas y formato del libro se concentran en las utilidades comunes de Excel.

La selección del generador puede representarse de forma simplificada de la siguiente manera:

```
1 if type == PRODUCT and category == ALTERNATIVE:
2     generate product alternative report
3 else if type == PRODUCT and category == INNOVATION:
4     generate product innovation report
5 else if type == KPI and category == ALTERNATIVE:
6     generate KPI alternative report
7 else if type == KPI and category == INNOVATION:
8     generate KPI innovation report
9 else:
10    reject invalid configuration
```

Listing 5.1: Selección del generador según los parámetros del informe

Flujo de generación de informes

El flujo completo de generación es el siguiente:

1. *ReportBatchController* recibe la petición *POST /api/reports*.
2. El controlador recibe los parámetros mediante *ReportRequestDto* y delega la coordinación en *ReportLaunchService*.
3. *ReportLaunchService* comprueba la posible duplicidad:

- *ReportDuplicateUtils* genera la firma asociada a la solicitud.
 - mediante el repositorio se comprueba si se ha desencadenado recientemente una solicitud con los mismos parámetros;
 - si existe una solicitud equivalente dentro de la ventana definida, no se inicia un nuevo job;
 - en caso contrario, se continúa con el lanzamiento.
4. *ReportLaunchService* utiliza *JobLauncher* para iniciar de forma asíncrona el job definido en Spring Batch.
 5. El job contiene un único step implementado mediante *GenerateReportTasklet*.
 6. Al comenzar la ejecución, *GenerateReportTasklet* extrae los parámetros del job y llama a *ReportDataProbeService.hasData()*.
 7. Si no existen datos para la combinación solicitada, el step finaliza con el estado *NO_DATA* sin construir el workbook.
 8. Si existen datos, el tasklet selecciona el generador correspondiente según el tipo y la categoría del informe.
 9. Los servicios especializados obtienen la información necesaria desde BIC Process Execution y Elasticsearch, realizan las transformaciones y cálculos y construyen el workbook utilizando Aspose Cells y las utilidades comunes.
 10. Una vez construido el fichero, *GenerateReportTasklet* delega su almacenamiento en *ReportService*.
 11. *ReportService* serializa el workbook, construye la entidad y utiliza *ReportRepository* para almacenar los metadatos y el contenido binario en PostgreSQL.
 12. El informe queda disponible para ser detectado mediante */api/reports/latest*, descargado mediante su identificador y consultado posteriormente desde el historial.

La lógica principal puede resumirse mediante el siguiente pseudocódigo:

```
1 requestGeneration(request):  
2   if reportDuplicateUtils.wasTriggeredRecently(request):  
3     return DUPLICATE  
4  
5   jobId = reportLaunchService.launch(request)  
6   return STARTED(jobId)  
7  
8
```

```

9 execute(stepContribution, jobParameters):
10     if not reportDataProbe.hasData(jobParameters):
11         stepContribution.setExitStatus(NO_DATA)
12         return FINISHED
13
14     workbook = generateWorkbook(jobParameters)
15     reportService.store(workbook)
16
17     return FINISHED

```

Listing 5.2: Pseudocódigo de la solicitud y generación de un informe

De este modo, *ReportLaunchService* coordina la comprobación de duplicidad y el lanzamiento del job, mientras que la disponibilidad de datos forma parte del propio flujo de ejecución y es comprobada por *GenerateReportTasklet*. Esta separación evita introducir en el controlador lógica específica de cada tipo de informe.

Comprobación de disponibilidad de datos

ReportDataProbeService permite determinar si existen datos suficientes para generar el informe antes de ejecutar toda la lógica de construcción del workbook.

Su implementación, *ReportDataProbeServiceImpl*, selecciona la estrategia de comprobación según el tipo y la categoría del informe.

Para los informes de Producto, se normalizan previamente las franquicias y las fechas mediante *ReportParamUtils*. En la categoría Alternativa se utiliza *AlternativeExportDataService*, mientras que para Innovación se utiliza *InnovationExportDataService*. Se considera que no existen datos cuando los procesos recuperados no contienen tareas relevantes.

Para KPI, las franquicias se transforman al formato esperado y se utiliza *KpiAlternativeInstanceService* o *KpiInnovationInstanceService* según la categoría. La comprobación se basa en la existencia de proyectos asociados a las franquicias solicitadas.

De forma simplificada:

```

1 hasData(type, category, franchises, startDate, endDate):
2     if type == PRODUCT:
3         normalize parameters and dates
4         data = load PRODUCT data for category
5         return data contains at least one task
6
7     if type == KPI:
8         franchiseList = parse franchises
9         data = load KPI projects for category
10        return data contains at least one project
11
12    raise invalid report type

```

Listing 5.3: Comprobación de disponibilidad de datos

Diseño del Job y Step de Spring Batch

El job de generación se define en *ReportBatchConfig* y utiliza:

- **Parámetros del job:** tipo, categoría, franquicias, fechas y los identificadores necesarios para la ejecución.
- **Un único Step:** implementado mediante *GenerateReportTasklet*.
- **Executor asíncrono:** configurado en *BatchAsyncConfiguration*, permitiendo que el job se ejecute sin bloquear el thread encargado de responder a la petición HTTP.

Se optó por un *Tasklet* en lugar del modelo *ItemReader–ItemProcessor–ItemWriter*, ya que la generación de cada informe constituye una operación completa que combina la obtención de información, el cálculo de métricas y la construcción de un workbook complejo.

Spring Batch proporciona además las tablas de metadatos necesarias para registrar las instancias, ejecuciones, parámetros y estados asociados a los jobs y steps.

Una vez generada la información, *GenerateReportTasklet* no accede directamente al repositorio. El almacenamiento se delega en *ReportService*, que centraliza la conversión del workbook a bytes, la construcción de la entidad y su persistencia mediante *ReportRepository*.

Esta decisión evita repetir el proceso de serialización y almacenamiento en cada uno de los generadores de informes.

5.2.3 Capa de datos

La capa de persistencia almacena los informes generados junto con la información necesaria para identificarlos, consultarlos y descargarlos posteriormente.

Modelo de persistencia

El esquema *aesla_reports* contiene una única tabla principal *report*, gestionada mediante migraciones de Flyway. La entidad JPA *ReportEntity* representa esta tabla.

Los principales atributos son:

- *id* (UUID): identificador único.
- *file_name* (String): nombre del fichero Excel.
- *report_type* (enum): KPI o PRODUCT.
- *report_category* (enum): INNOVATION o ALTERNATIVE.
- *created_at* (LocalDateTime): fecha y hora de creación.

- *start_date* y *end_date* (LocalDate): rango de fechas opcional.
- *franchises* (String): franquicias asociadas.
- *file_size* (Long): tamaño del fichero.
- *content* (byte[]): contenido binario del workbook.

Justificación del esquema y uso de Flyway

Se utiliza un esquema específico, *aesla_reports*, para mantener los datos del Report Service separados del resto de los esquemas utilizados por la plataforma.

Flyway permite:

- versionar los cambios del esquema mediante scripts [SQL](#);
- aplicar únicamente las migraciones todavía no ejecutadas;
- mantener trazabilidad mediante *flyway_schema_history*;
- integrar el proceso de migración con el arranque de Spring Boot.

La configuración desarrollada contempla tres grupos principales de migraciones. En primer lugar se crea el esquema *aesla_reports*; posteriormente se crea la tabla *report*; finalmente se incorporan las tablas de metadatos requeridas por Spring Batch.

Entre estas últimas se encuentran:

- *BATCH_JOB_INSTANCE*;
- *BATCH_JOB_EXECUTION*;
- *BATCH_JOB_EXECUTION_PARAMS*;
- *BATCH_STEP_EXECUTION*;
- *BATCH_JOB_EXECUTION_CONTEXT*;
- *BATCH_STEP_EXECUTION_CONTEXT*.

Estas tablas permiten a Spring Batch identificar las instancias de los jobs, registrar sus ejecuciones y mantener los parámetros y contextos asociados.

La configuración correspondiente se encuentra en *application.yaml*, donde se definen las propiedades relacionadas con JPA, JDBC, Flyway y la infraestructura de Spring Batch. El objetivo es garantizar que las estructuras necesarias se encuentren disponibles antes de iniciar cualquier job.

ReportRepository extiende *JpaRepository* y encapsula el acceso a los informes almacenados. Sobre él, *ReportService* implementa la lógica asociada a la persistencia y recuperación: serializa el workbook, construye la entidad, ejecuta la operación *save* y recupera los informes cuando son solicitados por identificador o por los criterios correspondientes.

Cuadro 5.2: Atributos de la entidad *Report* en *aesla_reports.report*.

Campo (columna)	Tipo lógico (SQL)	(Java / Nulo)	Descripción
<i>id</i>	UUID / UUID	No	Identificador único del informe (clave primaria).
<i>file_name</i>	String / varchar(512)	No	Nombre del fichero Excel generado.
<i>report_type</i>	enum ReportType / varchar	No	Tipo de informe: <i>KPI</i> o <i>PRODUCT</i> .
<i>report_category</i>	enum ReportCategory / varchar	No	Categoría: <i>INNOVATION</i> o <i>ALTERNATIVE</i> .
<i>created_at</i>	LocalDateTime / timestamp	No	Fecha y hora de creación.
<i>start_date</i>	LocalDate / date	Sí	Fecha inicial del rango solicitado.
<i>end_date</i>	LocalDate / date	Sí	Fecha final del rango solicitado.
<i>franchises</i>	String / text	No	Franquicia o conjunto de franquicias seleccionadas.
<i>file_size</i>	Long / bigint	No	Tamaño del fichero en bytes.
<i>content</i>	byte[] / bytea	No	Contenido binario del workbook.

5.3 Interacción cliente–servidor

Esta sección concreta las operaciones intercambiadas entre el cliente Angular y el backend Spring Boot a través de la API REST.

5.3.1 Endpoints REST

Inicio de una generación

El endpoint:

POST /api/reports

recibe los parámetros necesarios para iniciar una nueva generación. Un cuerpo de petición simplificado puede representarse de la siguiente manera:

```
1 {  
2   "reportType": "KPI",  
3   "reportCategory": "INNOVATION",  
4   "franchises": ["Starbucks", "Vips"],  
5   "startDate": "2024-01-01",  
6   "endDate": "2024-12-31"  
7 }
```

Listing 5.4: Ejemplo de solicitud de generación en JSON

La operación realiza las validaciones iniciales y la comprobación de duplicidad. Cuando la solicitud puede procesarse, se inicia el job asíncrono y se devuelve al cliente la confirmación de que la generación ha comenzado.

La ausencia de datos puede detectarse posteriormente durante la ejecución del tasklet, por lo que el cliente utiliza el mecanismo de seguimiento para conocer el resultado final.

Consulta del historial

El endpoint:

GET /api/reports/history?page=0&size=5

devuelve una página del historial de informes.

Una respuesta simplificada es:

```
1 {  
2   "content": [  
3     {  
4       "id": "uuid",  
5       "fileName": "KPI_Innovation_2024-11-15.xlsx",  
6       "reportType": "KPI",  
7       "reportCategory": "INNOVATION",  
8       "createdAt": "2024-11-15T10:30:00",  
9       "franchises": "Starbucks,Vips",  
10      "fileSize": 1048576  
11    }  
12  ],  
13  "totalElements": 42,  
14  "totalPages": 9,  
15  "number": 0  
16 }
```

Listing 5.5: Ejemplo de respuesta paginada en JSON

Seguimiento de la generación

El endpoint:

GET /api/reports/latest

se utiliza durante el mecanismo de *polling* para comprobar si el informe solicitado ya se encuentra disponible. El cliente realiza consultas periódicas hasta obtener la confirmación de disponibilidad o alcanzar el tiempo máximo de espera configurado.

Descarga de un informe

El endpoint:

GET /api/reports/{id}/download

recupera un informe almacenado a partir de su identificador. Cuando el informe existe, el servidor devuelve su contenido binario en formato Excel; si el identificador no corresponde a ningún informe almacenado, la operación devuelve el error correspondiente.

5.3.2 Manejo de errores

La API utiliza códigos [HTTP](#) estándar para representar el resultado de las operaciones. Entre los principales casos contemplados se encuentran:

- **200 OK:** consulta o descarga realizada correctamente.
- **202 Accepted:** petición aceptada para procesamiento asíncrono, cuando este código es utilizado por la implementación del controlador.
- **400 Bad Request:** parámetros de entrada inválidos.
- **409 Conflict:** existencia de una solicitud equivalente reciente cuando la duplicidad se representa mediante este código.
- **404 Not Found:** no existe el informe solicitado.
- **500 Internal Server Error:** error no controlado durante el procesamiento.

Cuando corresponde, las respuestas de error incluyen información estructurada que permite al cliente identificar la situación y mostrar un mensaje comprensible al usuario. Por ejemplo:

```
1 {  
2   "error": "DUPLICATE_REPORT",  
3   "message": "Un informe idéntico fue generado recientemente.",  
4   "reportId": "uuid"  
5 }
```

Listing 5.6: Ejemplo simplificado de una respuesta de error

Implementación y pruebas

6.1 Tareas de mejora

Durante el desarrollo del proyecto, se abordaron diversas tareas de mejora que no solo añadieron nuevas funcionalidades, sino que también optimizaron la calidad, seguridad y mantenibilidad del código existente. Estas tareas se clasificaron en diferentes categorías según su naturaleza y prioridad, y se gestionaron a través del sistema de seguimiento Jira como parte del [EPIC](#) "Improvements for AESLA Project in 2025 Q3 & Q4".

Las tareas de mejora se distribuyeron a lo largo de los tres meses, abordando aspectos críticos como la seguridad de las dependencias, la cobertura de pruebas, la refactorización del código, la observabilidad y la incorporación de nuevas funcionalidades. A continuación, se describen en detalle las principales categorías de tareas de mejora abordadas durante el proyecto.

6.1.1 Organización de los work items

El trabajo se organizó en Jira mediante el *EPIC Improvements for AESLA Project in 2025 Q3 & Q4*. Este elemento actuó como padre común de los work items relacionados con la evolución del Report Service. La mayoría fueron tareas técnicas, aunque también se registraron incidencias de tipo *Bug* y una historia de usuario de prioridad alta que concentró la introducción del procesamiento asíncrono con Spring Batch.

Cada work item se desarrolló en una rama independiente identificada con su número. En ella se realizaron los commits y, al finalizar, se abrió un Pull Request hacia *development*. El cambio se integraba después de superar la revisión de código y las comprobaciones de calidad y QA. Todos los elementos compartían la misma *fix version* del Report Service, facilitando la trazabilidad de los cambios incluidos en la publicación.

Entre los trabajos realizados se incluyeron la reducción de duplicación, el aumento de

cobertura, la persistencia y consulta de informes, la generación asíncrona, la incorporación de logs, la mejora de las páginas de generación e historial y la resolución de problemas de inicialización de Spring Batch.

6.1.2 Persistencia y consulta de informes

Una de las tareas principales consistió en permitir que los informes generados se almacenaran en la base de datos y pudieran descargarse posteriormente desde el historial. Aunque se gestionó como una tarea técnica, se especificó mediante una historia de usuario: un usuario debía poder acceder a la página de historial, consultar los informes generados y descargar de nuevo cualquiera de ellos.

La implementación se dividió en varias subtarear: creación del esquema de base de datos, integración de su gestión mediante Flyway, creación del repositorio para recuperar el contenido y los metadatos, y refactorización del tratamiento de franquicias y fechas en los informes de Producto y KPI para utilizar una aproximación común. La información almacenada incluye la fecha y hora de generación, el nombre y tamaño del fichero, su contenido y los datos necesarios para identificar la configuración utilizada.

6.1.3 Vulnerabilidades

El *frontend* presentaba inicialmente múltiples vulnerabilidades (CVE) asociadas a sus dependencias. En un primer momento, el origen del problema se debía en parte a la coexistencia de los gestores de paquetes *npm* y *Yarn*, lo que generaba inconsistencias en el árbol de dependencias y diferentes resoluciones de versiones según el gestor empleado. Una vez unificada la gestión en *Yarn*, las vulnerabilidades no desaparecieron por completo, ya que la versión de *Angular* utilizada (15) era considerablemente anterior, lo que limitaba el acceso a versiones parcheadas de algunas librerías, tal y como se recoge en la documentación técnica.

Estrategia adoptada:

1. **Estandarización en Yarn:** se eliminaron los ficheros *package-lock.json*, se configuró *Yarn* como gestor único en los archivos *angular.json* y *package.json*, y se realizó un proceso de deduplicación con *yarn-deduplicate* seguido de una reinstalación completa de dependencias. Esta medida permitió estabilizar el entorno de desarrollo y homogeneizar la instalación en local y en CI.
2. **Mitigación mediante *resolutions*:** dado que la actualización a una versión posterior de Angular con soporte vigente quedaba fuera del alcance del proyecto, se optó por emplear el bloque *resolutions* de *Yarn* para forzar versiones parcheadas de dependencias

transitivas, sobrescribiendo las versiones vulnerables solicitadas por los paquetes padre. Esta técnica permitió corregir gran parte de los avisos de seguridad sin alterar el núcleo del framework.

Situación final: tras la estandarización y las *resolutions*, el número de vulnerabilidades se redujo significativamente, quedando solo tres CVEs menores vinculadas a dependencias del tooling de *Angular*. Dado que su impacto era limitado y no afectaba al entorno de ejecución, se decidió priorizar la integración funcional completa entre el *frontend* y el *backend* actualizado con las nuevas funcionalidades anteriormente comentadas. Como trabajo futuro se plantea la actualización del proyecto a una versión posterior de Angular con soporte vigente, lo que permitiría eliminar el bloque de *resolutions* y resolver de forma nativa las vulnerabilidades remanentes.

6.1.4 Coverage

Una de las métricas de calidad más importantes establecidas por GBTEC en SonarCloud es la *Coverage* (cobertura de código) mediante pruebas unitarias. Al inicio del proyecto, el Report Service presentaba una cobertura del 17.7%, considerablemente por debajo del umbral mínimo del 20% exigido por la empresa para aprobar los PRs.

Para abordar esta situación, se implementó una estrategia sistemática de mejora de la cobertura de tests:

- **Identificación de código sin cobertura:** se analizaron los informes de SonarCloud para identificar las clases, métodos y ramas lógicas que carecían de pruebas unitarias.
- **Priorización de componentes críticos:** se priorizaron los servicios de negocio (*ReportService*, *BatchService*), repositorios (*ReportRepository*) y utilidades de generación Excel (*ExcelGenerator*).
- **Implementación de tests unitarios:** se escribieron nuevos tests utilizando JUnit 5 y Mockito para simular dependencias externas, cubriendo casos normales, casos límite y situaciones de error.
- **Tests de integración con Testcontainers:** se añadieron tests de integración que levantan contenedores efímeros de PostgreSQL para validar el comportamiento del repositorio y las migraciones Flyway.

Tras la implementación de las mejoras, la cobertura aumentó del 17.7% al 25.7%, lo que supone un incremento de 8 puntos porcentuales y permitió superar el umbral mínimo del 20% establecido por la empresa. Las capturas originales de SonarCloud utilizadas como evidencia se incluyen en el apéndice A.2.

6.1.5 Refactorización

La refactorización se centró en extraer utilidades comunes, homogeneizar la gestión de parámetros y reducir la lógica repetida entre los distintos tipos y categorías de informe. Estas modificaciones facilitaron la incorporación de nuevas funcionalidades y mejoraron la mantenibilidad del código.

6.1.6 Duplicación

Otra métrica crítica de calidad de código es el porcentaje de **Duplicación de código**, que mide la cantidad de código repetido en el proyecto. El código duplicado aumenta la deuda técnica, dificulta el mantenimiento y puede introducir inconsistencias cuando se realizan cambios que deben aplicarse en múltiples lugares.

Al inicio del proyecto, el análisis de SonarCloud reveló un 15.8% de duplicación en el código del Report Service. Este nivel de duplicación se debía principalmente a:

- **Lógica repetida en controladores:** métodos similares para manejar diferentes tipos de informes con código casi idéntico.
- **Generación de Excel duplicada:** múltiples implementaciones de formateo y estilo de celdas Excel para diferentes hojas del workbook.
- **Validaciones redundantes:** comprobaciones de parámetros y fechas replicadas en varios servicios.
- **Consultas a Elasticsearch similares:** queries con estructura parecida pero ligeras variaciones para diferentes variables de proceso.

Para reducir la duplicación, se aplicaron las siguientes técnicas de refactorización:

- **Extracción de métodos comunes:** se identificaron patrones repetidos y se extrajeron a métodos reutilizables en clases de utilidad.
- **Uso de herencia y composición:** se crearon clases base abstractas para generadores de informes, compartiendo lógica común y permitiendo especialización mediante herencia.
- **Parametrización de lógica:** se convirtieron bloques de código duplicados en métodos parametrizados que reciben configuración mediante argumentos.
- **Patrón Template Method:** se aplicó este patrón de diseño para definir el esqueleto de algoritmos de generación, delegando pasos específicos a subclasses.

- **Builders y factories:** se implementaron patrones Builder para construcción de objetos complejos (configuración Excel, parámetros de jobs) reduciendo código repetitivo.

Como resultado de las tareas de refactorización, la duplicación de código se redujo del 15.8% al 10.0%, lo que supone una disminución de 5.8 puntos porcentuales. Esta evolución refleja una reducción cuantificable de la lógica repetida detectada por SonarCloud. Las evidencias originales se incluyen en el apéndice [A.2](#).

Cuadro 6.1: Evolución de las principales métricas de calidad medidas con SonarCloud.

Métrica	Valor inicial	Valor final	Variación
Cobertura de código	17.7%	25.7%	+8.0 pp
Duplicación de código	15.8%	10.0%	-5.8 pp

6.1.7 Generación asíncrona con Spring Batch

La historia de usuario de mayor alcance, y de prioridad alta, tuvo como objetivo que el usuario pudiera solicitar un informe sin tener que permanecer esperando activamente durante toda la descarga. Para ello, se implementó la lógica de generación mediante Spring Batch, se adaptó la API REST y se incorporaron notificaciones para informar de que el proceso había comenzado y de los posibles resultados.

La generación se ejecuta en segundo plano y el informe se almacena en el esquema *aesla_reports*. Tras recibir la confirmación de inicio, el frontend realiza *polling* sobre */api/reports/latest* cada cinco segundos, utilizando los parámetros del informe para identificar la generación correspondiente. Cuando el backend responde con *ready=true* y un identificador, el cliente solicita automáticamente la descarga mediante */api/reports/{id}/download*. El mecanismo tiene un tiempo máximo de espera de 120 segundos; si se supera, se informa al usuario de que la generación está tardando más de lo esperado y el informe puede consultarse posteriormente en Historial. Los casos de ausencia de datos, errores de red y duplicidad se comunican mediante mensajes específicos.

6.1.8 Observabilidad y soporte

Para facilitar el soporte al cliente se incorporaron logs informativos en puntos relevantes del proceso de exportación a Excel. Estos registros permiten identificar el avance de la generación y localizar con mayor rapidez posibles fallos en los servicios. Como parte de una mejora menor adicional, se reubicó un bloque *try-catch* en dos servicios y se añadió el registro correspondiente a nivel informativo.

6.1.9 Mejoras de la interfaz

Se desarrollaron y ajustaron las páginas de generación e historial. La primera incorpora la configuración de los parámetros, las validaciones y las notificaciones asociadas al proceso. La segunda permite consultar los informes almacenados, visualizar su fecha de generación, nombre, tamaño y contenido disponible, y descargarlos posteriormente.

6.1.10 Incidencias y trabajo no realizado

Durante la integración de Spring Batch se detectó una incidencia relacionada con la inicialización del esquema y de las tablas de Spring Batch. La configuración inicial no creaba correctamente estos elementos mediante Flyway durante el arranque, lo que provocaba fallos al iniciar o ejecutar el procesamiento. La incidencia se resolvió ajustando la configuración de las migraciones para que las tablas necesarias estuvieran disponibles antes de lanzar los trabajos.

También se planteó una historia de usuario para permitir la eliminación de informes desde el historial. Esta funcionalidad no llegó a implementarse dentro del alcance del proyecto y, por tanto, se mantiene como una ampliación futura.

6.2 Pruebas

La estrategia de validación se organizó en tres niveles complementarios: pruebas unitarias sobre la lógica de negocio y las utilidades, pruebas de integración sobre la persistencia y las migraciones, y pruebas manuales funcionales sobre los flujos completos en el entorno de QA. El objetivo fue comprobar tanto el comportamiento aislado de los componentes como la integración entre frontend, backend y capa de persistencia.

6.2.1 Relación entre requisitos y pruebas

Las pruebas se definieron tomando como referencia los requisitos funcionales descritos en el capítulo 4. La tabla 6.2 muestra la relación entre cada requisito y los principales mecanismos utilizados para validarlo.

Cuadro 6.2: Trazabilidad entre requisitos funcionales y pruebas realizadas.

Requisito	Comportamiento validado	Tipo de prueba	Casos
RF-01	Generación asíncrona, validación de parámetros, tratamiento de ausencia de datos y seguimiento de la generación	Unitarias, integración y QA	CP-01, CP-02, CP-06, CP-07, CP-08
RF-02	Detección de solicitudes equivalentes dentro de la ventana temporal configurada	Unitarias y QA	CP-03
RF-03	Persistencia del fichero y descarga o carga posterior	Integración y QA	CP-05, CP-08
RF-04	Consulta del historial, ordenación y presentación de los informes almacenados	Integración y QA	CP-04

6.2.2 Catálogo representativo de pruebas

La tabla 6.3 recoge una selección de los casos funcionales más representativos ejecutados durante el proyecto. Se indica como *OK* cuando el comportamiento observado coincidió con el resultado esperado.

Cuadro 6.3: Casos de prueba representativos del Report Service.

ID	Caso de prueba	Resultado esperado	Resultado
CP-01	Generar un informe KPI de Innovación con datos disponibles	El job se ejecuta y el informe queda almacenado	OK
CP-02	Generar un informe sin datos para los parámetros solicitados	El job finaliza con el estado <i>NO_DATA</i> sin generar el Excel	OK
CP-03	Repetir una misma solicitud dentro de la ventana de cinco minutos	Se detecta la duplicidad y no se inicia una segunda generación	OK
CP-04	Consultar el historial de informes	Los informes se muestran ordenados de más reciente a más antiguo	OK
CP-05	Descargar un informe previamente almacenado	Se recupera correctamente el fichero Excel asociado al identificador	OK
CP-06	Introducir un rango de fechas inválido	La solicitud es rechazada y se informa del error de validación	OK
CP-07	Consultar <i>/api/reports/latest</i> mientras continúa una generación	Se devuelve <i>ready=false</i> mientras el informe no está disponible	OK
CP-08	Detectar mediante <i>polling</i> que el informe está disponible	Se obtiene el identificador y el frontend inicia la descarga del fichero	OK

Estos casos no pretenden constituir un catálogo exhaustivo, sino recoger los escenarios más relevantes desde el punto de vista funcional y de integración.

6.2.3 Pruebas manuales y entorno de QA

Para cada tarea asignada en Jira se siguió un flujo de validación manual en un entorno de pruebas controlado antes de integrar los cambios en las ramas principales. El proceso de QA se apoyaba en la infraestructura de Ansible gestionada mediante AWX y en el registro de imágenes Quay.

Flujo de trabajo de QA:

1. **Desarrollo y commits:** tras completar la implementación de una funcionalidad o corrección, se realizaban los commits necesarios en la rama correspondiente.
2. **Actualización de versión:** se actualizaba el número de versión en los archivos de configuración del servicio modificado para mantener la trazabilidad.
3. **Build y publicación de imagen:** el pipeline de GitHub Actions construía automáticamente una nueva imagen Docker y la publicaba en Quay con la etiqueta correspondiente.
4. **Despliegue mediante AWX:** se ejecutaba el playbook de Ansible correspondiente para desplegar la versión en el entorno aislado de QA junto con los servicios necesarios.
5. **Validación funcional:** desde Google Chrome se comprobaban los principales flujos de la aplicación: generación con distintas combinaciones de parámetros, ausencia de datos, control de duplicidad, consulta del historial, descarga, mensajes de error y seguimiento mediante *polling*.
6. **Iteración o cierre:** si se detectaba una incidencia, se corregía y se repetía el proceso. Cuando el comportamiento era el esperado, la tarea se marcaba como *QA OK* y se continuaba con el proceso de integración mediante Pull Request.

La combinación de pruebas automatizadas y validación funcional en QA permitió comprobar los principales flujos antes de su integración en *development*.

Solución desarrollada

7.1 Introducción

En este capítulo se describe la solución desarrollada durante el proyecto, que comprende la modernización del backend, la persistencia de informes, el procesamiento asíncrono y la mejora de la interfaz de usuario del Report Service. A diferencia del sistema original, que ofrecía únicamente una pantalla básica de generación de informes, la solución final incorpora funcionalidades que mejoran la experiencia del usuario y las capacidades del sistema.

Las principales mejoras implementadas incluyen:

- Una interfaz renovada para el generador de informes con actualización periódica del estado mediante *polling*.
- Un módulo completamente nuevo de historial que permite consultar y redescargar informes previamente generados.
- Validaciones mejoradas y mensajes informativos que guían al usuario durante todo el flujo de trabajo.
- Control de duplicidad integrado que evita la regeneración innecesaria de informes idénticos.

A continuación, se detallan las páginas que componen la solución desarrollada.

7.2 Página de Generar

La página de generación de informes constituye el núcleo de la interfaz de usuario. Esta pantalla permite al usuario configurar todos los parámetros necesarios para crear un informe

personalizado y recibir retroalimentación inmediata sobre las validaciones, el inicio de la generación y la disponibilidad del informe.

En una primera fase de diseño, se había planteado la existencia de tres interfaces principales: una pantalla inicial a modo de menú de inicio desde la que el usuario podría acceder tanto al historial de informes como a la página de generación. Sin embargo, tras una discusión dentro del equipo, se concluyó que dicha estructura añadía una capa innecesaria de navegación y no aportaba valor real al flujo de uso. Se decidió, por tanto, simplificar la experiencia del usuario manteniendo únicamente dos interfaces —*Generar informe* e *Historial*—, siendo la primera la principal y punto de entrada de la aplicación.

7.2.1 Funcionalidad

- Permite configurar los parámetros del informe.
- Muestra notificaciones durante el inicio y el seguimiento periódico de la generación.
- Descarga del informe una vez finalizada la generación.
- Incluye un botón para navegar al historial.

7.2.2 Diseño de la interfaz

La interfaz del generador se ha diseñado para minimizar la incertidumbre del usuario: los controles relevantes se deshabilitan mientras una generación está en curso, se muestran mensajes claros sobre el estado del proceso y se ofrece ayuda contextual para elegir parámetros. Cuando el sistema detecta un informe idéntico reciente, la interfaz sugiere al usuario descargarlo desde Historial en lugar de volver a generarlo, respetando la política de control de duplicidad del backend.

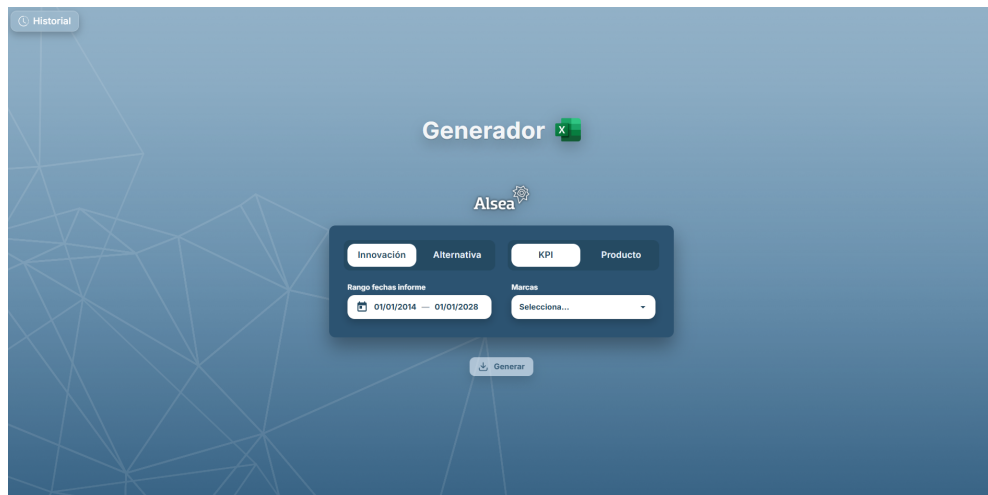


Figura 7.1: Interfaz principal del módulo de generación de informes.

7.2.3 Flujo de interacción

El flujo de trabajo implementado en la página de generación garantiza una experiencia coherente y eficiente. El siguiente diagrama ilustra el proceso completo desde que el usuario accede a la página hasta que puede redescargar el informe desde el historial.

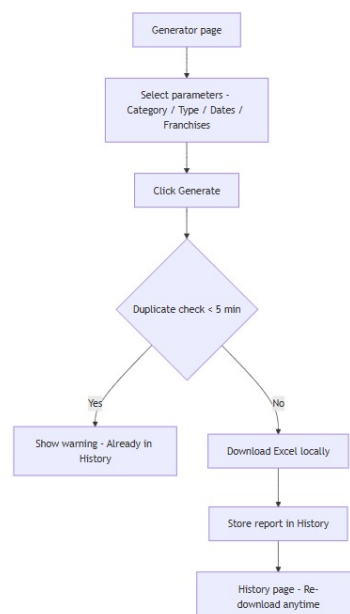


Figura 7.2: Diagrama de flujo del proceso de generación de informes.

Como se observa en la figura 7.2, el proceso comienza cuando el usuario selecciona los

parámetros del informe y pulsa el botón *Generar*. En ese momento, el sistema comprueba si existe un informe idéntico generado en los últimos 5 minutos. Si se detecta un duplicado, el sistema consulta `/api/reports/latest` y muestra un aviso indicando si el informe ya está disponible o si la generación continúa en curso. En caso contrario, se inicia el job asíncrono y el frontend comienza a consultar periódicamente `/api/reports/latest`. Cuando la respuesta indica que el informe está listo, el cliente solicita su descarga automáticamente; en cualquier caso, el informe queda también disponible en Historial.

7.3 Página de Historial

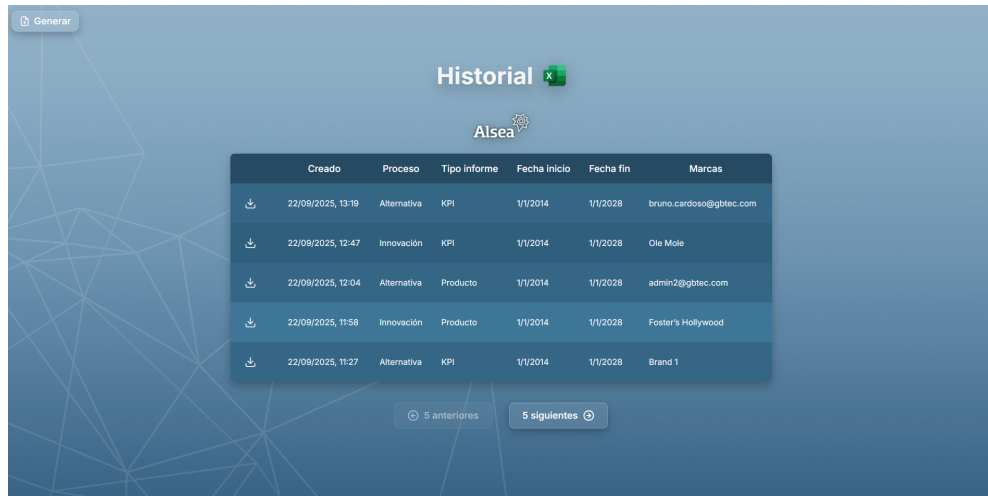
La página de historial es una funcionalidad completamente nueva desarrollada en este proyecto. Antes de esta mejora, el sistema no contaba con ningún mecanismo para consultar informes previamente generados, obligando a los usuarios a regenerarlos cada vez que necesitaban acceder a información pasada.

7.3.1 Funcionalidad

- Presenta una tabla con todos los informes generados.
- Cada fila incluye columnas de descarga, fecha de creación, tipo, categoría, franquicia(s), fechas y tamaño.
- Implementa paginación (*5 anteriores / 5 siguientes*).
- Los informes se ordenan de los más recientes a los más antiguos.
- Permite la redescarga directa de cualquier informe almacenado.

7.3.2 Diseño de la interfaz

El diseño de la tabla de historial prioriza la claridad y la facilidad de uso. Cada informe muestra toda la información relevante de forma estructurada, permitiendo al usuario identificar rápidamente el documento que necesita. El botón de descarga en cada fila facilita el acceso inmediato al contenido sin necesidad de regenerar el informe.



Creado	Proceso	Tipo informe	Fecha inicio	Fecha fin	Marcas
22/09/2025, 13:19	Alternativa	KPI	1/1/2014	1/1/2028	bruno.cardoso@gbtec.com
22/09/2025, 12:47	Innovación	KPI	1/1/2014	1/1/2028	Ole Mole
22/09/2025, 12:04	Alternativa	Producto	1/1/2014	1/1/2028	admin2@gbtec.com
22/09/2025, 11:58	Innovación	Producto	1/1/2014	1/1/2028	Foster's Hollywood
22/09/2025, 11:27	Alternativa	KPI	1/1/2014	1/1/2028	Brand 1

Figura 7.3: Vista de la página de historial de informes generados.

7.4 Release y publicación

Una vez completado el desarrollo de las funcionalidades descritas, con las interfaces validadas y aprobadas por el equipo, y habiendo superado todas las pruebas de QA en el entorno de pruebas gestionado mediante AWX, el último paso consistía en preparar y publicar la nueva versión del Report Service siguiendo el proceso formal de release establecido por la empresa.

Este proceso seguía las convenciones de Gitflow y las políticas de versionado de GBTEC, garantizando la trazabilidad completa y minimizando riesgos de despliegue:

1. **Creación de la rama release candidate (RC):** desde la rama *development*, que contenía todos los cambios validados e integrados, se creaba una nueva rama *release/vX.Y.Z-rc* (por ejemplo, *release/v2.1.0-rc*). Esta rama servía como candidata para la release y permitía realizar ajustes finales sin bloquear el desarrollo de nuevas funcionalidades en *development*.
2. **Version bump:** se actualizaban los números de versión en los archivos de configuración:
 - Backend: actualización del elemento `<version>` en el archivo *pom.xml* de Maven.

Estos cambios se comprometían en la rama RC con un mensaje descriptivo (p. ej., *"chore: bump version to 2.1.0"*).

3. **Builds y validaciones finales:** se ejecutaban los pipelines de GitHub Actions para compilar el proyecto completo, ejecutar todas las pruebas (unitarias e integradas) y verificar las métricas de calidad en SonarCloud. Se realizaba una última ronda de validaciones manuales en el entorno de QA para confirmar que no existían regresiones.
4. **Merge a *main*:** se creaba un PR desde la rama RC hacia la rama *main*, que representaba el estado estable de producción. Tras la revisión final y aprobación por parte del equipo, se integraban los cambios mediante merge commit preservando el historial completo.
5. **Creación del tag Git:** una vez completado el merge a *main*, se creaba un tag Git con el número de versión (e.g., *v2.1.0*) apuntando al commit de merge en la rama *main*. Este tag marcaba formalmente la versión publicable y permitía rastrear exactamente qué código se desplegó en producción.
6. **Merge a *development*:** paralelamente, se integraban los cambios de la rama RC de vuelta a *development* para sincronizar cualquier ajuste realizado durante el proceso de release (como el version bump o correcciones de última hora), garantizando que *development* reflejara siempre el estado más actualizado.
7. **Documentación:** se actualizaban los archivos README con notas de la release, se documentaban los cambios principales (changelog) y se comunicaba al cliente la disponibilidad de la nueva versión con sus mejoras y funcionalidades.

Como parte del cierre del proyecto, también se revisó y amplió de forma exhaustiva la documentación de los repositorios frontend y backend. Antes de esta actualización, los README contenían información limitada sobre la configuración y la ejecución del servicio. La nueva documentación incorporó instrucciones de puesta en marcha, descripción de la configuración necesaria, dependencias, comandos habituales, estructura general y consideraciones para el despliegue, facilitando la incorporación de nuevos desarrolladores y el mantenimiento posterior.

Con este proceso, la nueva versión del Report Service quedó lista para su uso en producción, incorporando todas las mejoras de almacenamiento histórico, procesamiento asíncrono, control de duplicidad y experiencia de usuario modernizada.

Conclusión y trabajo futuro

8.1 Conclusiones

Este Trabajo de Fin de Grado ha abordado la modernización del módulo *Report Service* integrado en la plataforma BIC para AESLA, dentro de un proyecto desarrollado durante tres meses de prácticas en GBTEC Software AG. Las tareas realizadas permitieron cubrir los principales objetivos planteados al inicio del proyecto y ampliar tanto las capacidades funcionales como la mantenibilidad del servicio.

Entre los principales resultados obtenidos destacan:

- **Persistencia e historial de informes:** se incorporó un almacenamiento persistente en PostgreSQL, gestionado mediante migraciones Flyway. Los informes generados pueden conservarse junto con sus metadatos y recuperarse posteriormente desde la nueva página de historial, evitando la necesidad de regenerarlos para volver a acceder a ellos.
- **Procesamiento asíncrono:** la generación se integró con Spring Batch, desacoplando el procesamiento del ciclo principal de la petición HTTP. El frontend realiza un seguimiento periódico mediante *polling*, permitiendo mantener la interfaz disponible mientras el informe se genera.
- **Control de duplicidad:** se incorporó una comprobación de solicitudes equivalentes dentro de una ventana temporal de cinco minutos, evitando iniciar una nueva generación cuando ya existe una solicitud reciente con los mismos parámetros.
- **Evolución de la interfaz:** se añadió una página específica de historial, nuevas validaciones y mensajes de estado asociados a la generación. Estas modificaciones proporcionan al usuario más información sobre el proceso y nuevas posibilidades de consulta y redescarga.

- **Mejora de las métricas de calidad:** la cobertura de código aumentó del 17.7% al 25.7%, superando el umbral mínimo del 20% definido por la empresa. Paralelamente, la duplicación detectada por SonarCloud se redujo del 15.8% al 10.0%.
- **Gestión de dependencias y vulnerabilidades:** la estandarización del gestor de paquetes en Yarn y el uso controlado de *resolutions* permitieron reducir los avisos de seguridad asociados a las dependencias, aunque permanecieron tres vulnerabilidades menores vinculadas al *tooling* de la versión utilizada de Angular.

Desde el punto de vista formativo, el proyecto permitió trabajar sobre una aplicación empresarial existente, donde las decisiones debían considerar no solo la implementación de nuevas funcionalidades, sino también la compatibilidad con código previo, la calidad, las pruebas, la seguridad y el proceso de despliegue. El uso de Jira, Git, Pull Requests, revisión de código, pipelines de integración continua y entornos de QA proporcionó una visión práctica del ciclo de desarrollo de software en un entorno profesional.

En conjunto, la solución desarrollada amplía de forma significativa las capacidades del Report Service respecto a su estado inicial y establece una base más mantenible para futuras evoluciones. Los resultados obtenidos responden a los objetivos funcionales definidos para el proyecto, aunque existen limitaciones y líneas de mejora que se detallan a continuación.

8.2 Limitaciones

Aunque los objetivos principales del proyecto fueron alcanzados, el alcance temporal y el contexto de desarrollo condicionaron algunas de las decisiones adoptadas. Las principales limitaciones identificadas son las siguientes:

- **Ausencia de pruebas formales de carga:** se validó funcionalmente la generación asíncrona y la ejecución de los principales flujos, pero no se realizó un estudio específico de rendimiento bajo concurrencia elevada. Por tanto, no se dispone de métricas que permitan cuantificar el comportamiento del sistema ante un número elevado de generaciones simultáneas.
- **Versión del frontend:** el proyecto permanece sobre Angular 15. Aunque se mitigaron gran parte de los avisos de seguridad asociados a las dependencias, quedaron tres vulnerabilidades menores relacionadas con el *tooling* de esta versión.

- **Ausencia de una política de retención:** los informes almacenados permanecen actualmente en la base de datos sin un mecanismo automático de eliminación o una política de conservación configurable.
- **Seguimiento de la generación:** el estado se consulta mediante *polling* periódico. Aunque este mecanismo permite detectar cuándo el informe está disponible, no proporciona un porcentaje de progreso ni información detallada sobre las distintas etapas internas de generación.
- **Evaluación de usabilidad:** las interfaces fueron validadas funcionalmente dentro del proceso de QA, pero no se realizó un estudio formal de usabilidad con usuarios finales ni se recopilieron métricas cuantitativas de experiencia de usuario.

Estas limitaciones no impiden el funcionamiento de las funcionalidades desarrolladas, pero delimitan el alcance de las conclusiones que pueden extraerse y permiten identificar posibles líneas de evolución del sistema.

8.3 Posibles ampliaciones futuras

Aunque el proyecto ha logrado cumplir con los objetivos planteados, existen diversas líneas de trabajo que podrían abordarse en el futuro para seguir mejorando el Report Service. Estas ampliaciones se han identificado durante el desarrollo del proyecto y representan oportunidades para aumentar la funcionalidad, seguridad y usabilidad del sistema.

8.3.1 Actualización de la versión de Angular

Como se indicó en el apartado de vulnerabilidades y en las limitaciones del proyecto, una de las principales mejoras pendientes consiste en migrar Angular 15 a una versión posterior compatible con el resto del stack y que disponga de soporte vigente.

Esta actualización permitiría:

- Reevaluar el árbol completo de dependencias del frontend.
- Reducir o eliminar, cuando sea posible, el bloque *resolutions* utilizado actualmente en *package.json*.
- Analizar nuevamente las vulnerabilidades residuales y utilizar versiones corregidas de las dependencias cuando estén disponibles.

- Beneficiarse de las mejoras introducidas en versiones posteriores del framework y de Angular Material.
- Reducir las incompatibilidades entre dependencias heredadas del proyecto.

La migración debería realizarse de forma progresiva, siguiendo las guías oficiales de actualización y validando el comportamiento de la aplicación tras cada cambio de versión. Debido a las posibles incompatibilidades entre dependencias, sería necesario acompañar la actualización de pruebas funcionales y de regresión.

8.3.2 Mejoras en la gestión del historial

Una de las funcionalidades más solicitadas para futuras versiones es la ampliación de las capacidades de gestión del historial de informes. Específicamente, se ha identificado la necesidad de implementar:

Eliminación de informes

Actualmente, todos los informes generados se almacenan indefinidamente en la base de datos, sin una política de retención explícita. Sería deseable permitir a los usuarios eliminar informes que ya no necesiten, liberando espacio de almacenamiento y facilitando la gestión del historial.

El diseño preliminar de esta funcionalidad incluiría:



The mockup shows a table titled 'Alsea' with the following columns: Creado, Proceso, Tipo informe, Fecha inicio, Fecha fin, and Marcas. Each row represents a report and includes a delete button (trash icon) in the first column.

	Creado	Proceso	Tipo informe	Fecha inicio	Fecha fin	Marcas
 	30/09/2025, 14:32	Alternativa	KPI	1/1/2014	1/1/2028	admin2@gbtec.com, admin@gbtec.com, Brand 1, bruno.cardoso@gbtec.com
 	30/09/2025, 14:32	Innovación	KPI	1/1/2014	1/1/2028	Domino's Pizza
 	30/09/2025, 10:55	Alternativa	Producto	1/1/2025	1/6/2025	Brand 1
 	30/09/2025, 10:54	Alternativa	KPI	1/1/2025	1/6/2025	admin2@gbtec.com
 	30/09/2025, 10:54	Innovación	KPI	1/1/2025	1/6/2025	Domino's Pizza

Figura 8.1: Mockup de la interfaz de historial con funcionalidad de eliminación de informes.

Como se observa en la figura 8.1, se añadiría una columna adicional con botones de eliminación para cada informe. Al pulsar el botón de borrar, aparecería un diálogo de confirmación para evitar eliminaciones accidentales.

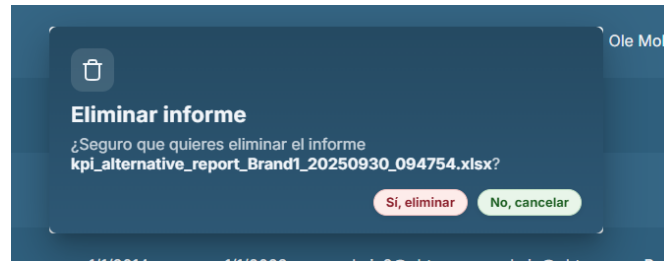


Figura 8.2: Diálogo de confirmación para la eliminación de informes.

La figura 8.2 muestra el diseño del diálogo de confirmación que solicitaría al usuario verificar la acción antes de eliminar permanentemente un informe del sistema.

Otras mejoras planificadas para el historial

Además de la funcionalidad de eliminación, se han identificado otras mejoras deseables:

- **Selección múltiple:** implementar checkboxes para seleccionar varios informes y realizar acciones en lote (eliminar múltiples informes, descargar varios informes simultáneamente).
- **Filtros avanzados:** añadir filtros por categoría, tipo, franquicia, rango de fechas, etc., para facilitar la búsqueda de informes específicos en historiales extensos.
- **Ordenación personalizable:** permitir ordenar la tabla por diferentes columnas (fecha, tamaño, tipo, categoría).
- **Exportación del listado:** posibilidad de exportar el listado del historial a CSV o Excel para auditorías o análisis externos.

8.3.3 Refactorización adicional del código backend

Aunque durante el proyecto se realizaron importantes tareas de refactorización que mejoraron las métricas de calidad del código, siempre existe margen de mejora. Las áreas identificadas para futuras mejoras incluyen:

- Mayor modularización de utilidades comunes.
- Ampliación de la cobertura de pruebas unitarias e integradas.
- Optimización de consultas a la base de datos.
- Implementación de patrones de diseño adicionales para mejorar la mantenibilidad.

8.3.4 Funcionalidades avanzadas

Mirando más allá de las mejoras inmediatas, se podrían considerar funcionalidades más ambiciosas:

- **Programación de informes:** permitir a los usuarios programar la generación automática de informes periódicos.
- **Notificaciones:** enviar notificaciones cuando un informe esté listo.
- **Plantillas personalizables:** permitir a los usuarios definir plantillas de informe con métricas específicas.
- **Dashboard analítico:** crear visualizaciones interactivas de las métricas de los informes sin necesidad de descargar Excel.
- **Control de acceso granular:** implementar permisos específicos por usuario o rol para la generación y consulta de informes.

Estas ampliaciones futuras representan oportunidades para seguir evolucionando el Report Service hacia una herramienta cada vez más completa y alineada con las necesidades cambiantes de AESLA.

Apéndice

Evidencias técnicas

A.1 Arquitectura detallada

La figura [A.1](#) amplía la representación simplificada presentada en el capítulo 5 e incorpora los principales componentes internos del frontend y del backend, el procesamiento asíncrono mediante Spring Batch, las fuentes de datos y los elementos de infraestructura utilizados durante el despliegue.

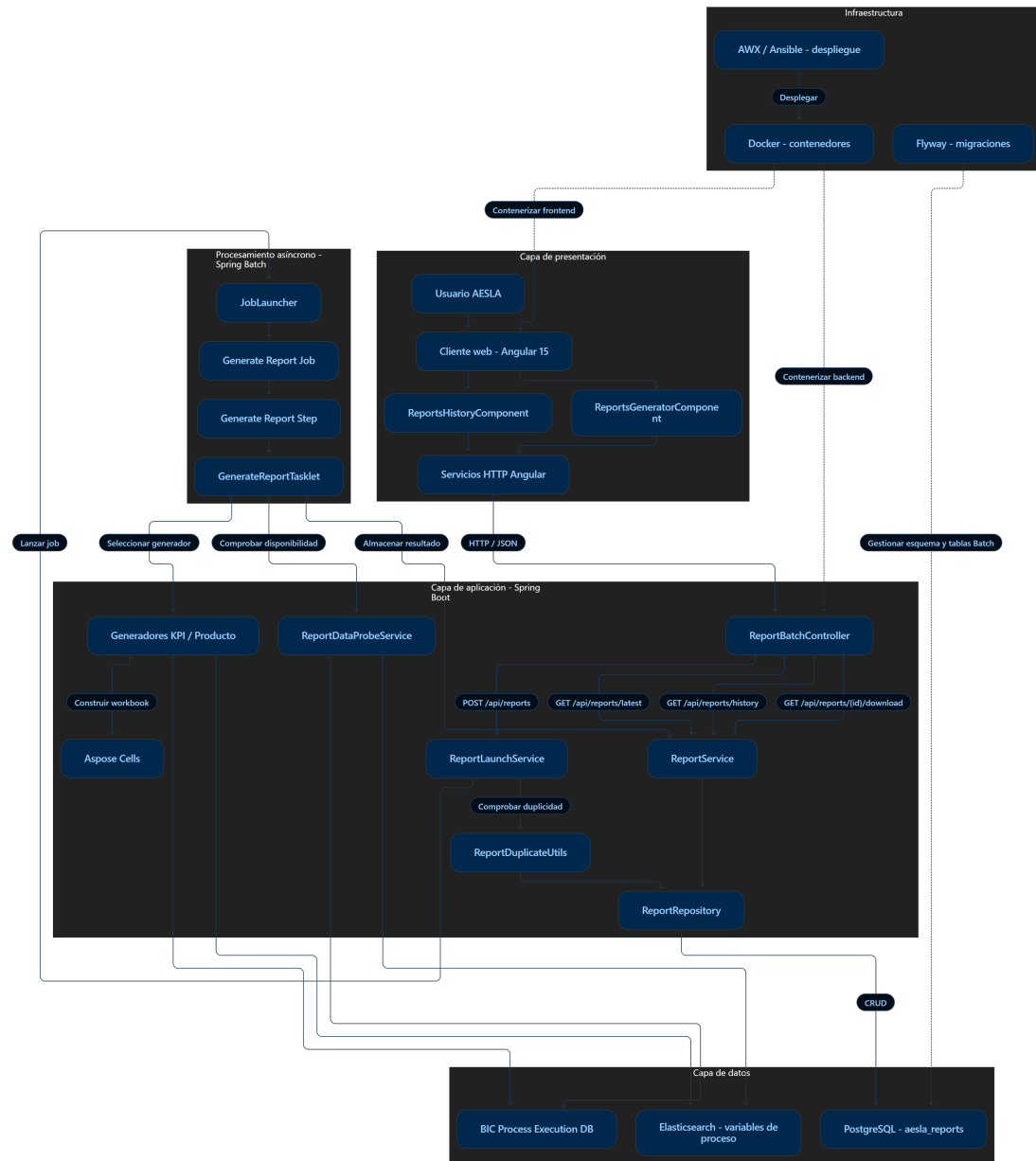


Figura A.1: Arquitectura detallada del Report Service. Fuente: elaboración propia.

En la capa de presentación, el usuario interactúa con los componentes Angular de generación e historial, que centralizan sus comunicaciones con el backend a través de servicios HTTP. El punto de entrada en la capa de aplicación es *ReportBatchController*, encargado de exponer las operaciones REST utilizadas por el cliente.

Para una nueva generación, el controlador delega en *ReportLaunchService*, que realiza la comprobación previa de duplicidad antes de iniciar el job de Spring Batch. El procesamiento asíncrono se articula mediante *JobLauncher*, un job, un step y *GenerateReportTasklet*. Durante

su ejecución se comprueba la disponibilidad de datos mediante *ReportDataProbeService* y se selecciona la lógica de generación correspondiente al tipo y categoría de informe.

Los datos utilizados proceden de BIC Process Execution y Elasticsearch, mientras que Aspose Cells proporciona las operaciones necesarias para la construcción de los libros Excel. Finalmente, *ReportService* centraliza el almacenamiento del resultado y utiliza *ReportRepository* para persistir el contenido y sus metadatos en el esquema *aesla_reports* de PostgreSQL.

La infraestructura complementaria incluye Flyway para gestionar las migraciones de base de datos, Docker para la contenerización de los servicios y AWX/Ansible para automatizar su despliegue en los diferentes entornos.

A.2 Evidencias de calidad de código

Las siguientes figuras recogen las capturas de SonarCloud utilizadas como evidencia de las métricas de cobertura y duplicación presentadas en el capítulo 6. En el cuerpo principal de la memoria se muestran únicamente los valores iniciales y finales, mientras que las capturas completas se trasladan a este apéndice para facilitar su consulta sin sobrecargar el capítulo de implementación y pruebas.



Figura A.2: Captura de SonarCloud correspondiente a la cobertura inicial del proyecto: 17.7%.



Figura A.3: Captura de SonarCloud correspondiente a la cobertura final del proyecto: 25.7%.



Figura A.4: Captura de SonarCloud correspondiente a la duplicación inicial del proyecto: 15.8%.



Figura A.5: Captura de SonarCloud correspondiente a la duplicación final del proyecto: 10.0%.

Lista de acrónimos

- API** Application Programming Interface. [11](#), [24](#)
- AWX** Interfaz web para la automatización con Ansible. [11](#)
- BPM** Business Process Management. [2](#)
- CI** Continuous Integration. [11](#)
- CSS** Cascading Style Sheets. [11](#)
- EAM** Enterprise Architecture Management. [2](#)
- EPIC** Conjunto de historias de usuario agrupadas en Jira. [12](#), [13](#)
- GRC** Governance, Risk and Compliance. [2](#)
- HTML** HyperText Markup Language. [11](#)
- HTTP** HyperText Transfer Protocol. [20](#), [24](#)
- HTTPS** HyperText Transfer Protocol Secure. [3](#)
- JPA** Java Persistence API. [9](#)
- JSON** JavaScript Object Notation. [9](#), [24](#)
- KPI** Key Performance Indicator. [2](#)
- PR** Pull Request. [11](#), [13](#)
- QA** Quality Assurance. [11](#), [13](#)

REST Representational State Transfer. [19](#), [24](#)

SQL Structured Query Language. [9](#)

UUID Universally Unique Identifier. [24](#)

Glosario

Angular Framework de desarrollo web basado en TypeScript para la construcción de aplicaciones cliente (*frontend*) modulares y escalables. [10](#)

Ansible Herramienta de automatización de configuración y despliegue de aplicaciones que utiliza archivos YAML para definir tareas y flujos de trabajo. [11](#)

Aspose Cells Biblioteca de Java empleada para la generación y manipulación avanzada de archivos en formato Excel. [9](#)

Code Smells Síntomas superficiales en el código fuente que pueden indicar problemas más profundos de diseño o implementación. Aunque no impiden el funcionamiento del software, los code smells dificultan la comprensión, el mantenimiento y la evolución del código. [11](#)

Coverage Métrica de calidad de software que mide el porcentaje de código fuente ejecutado durante las pruebas automatizadas. Un mayor coverage indica que una proporción más alta del código ha sido verificada mediante tests. [11](#)

Docker Plataforma de contenedores que permite empaquetar aplicaciones y sus dependencias en un entorno aislado y reproducible. [13](#)

Duplicación de código Métrica que indica el porcentaje de código duplicado o repetido en un proyecto. La duplicación excesiva dificulta el mantenimiento, ya que los cambios deben replicarse en múltiples lugares, aumentando el riesgo de inconsistencias. [11](#)

Elasticsearch Motor de búsqueda y análisis distribuido basado en documentos JSON. Permite realizar búsquedas complejas y agregaciones de datos en tiempo real. [2](#)

Gitflow Convención y conjunto de prácticas para el uso de ramas en Git que organiza el desarrollo mediante ramas *feature*, *release*, *hotfix* y *develop/main*, facilitando la gestión de releases y mantenimiento. [11](#)

GitHub Actions Plataforma de integración continua y entrega continua (CI/CD) integrada en GitHub que permite definir flujos de trabajo (workflows) para compilar, probar y desplegar aplicaciones mediante acciones reutilizables. [11](#)

Heatmap Representación gráfica de datos en la que los valores se representan mediante colores, facilitando la identificación de patrones y tendencias. [2](#)

Histograma Representación gráfica de la distribución de datos mediante barras verticales u horizontales, donde cada barra representa la frecuencia o cantidad de elementos en un rango o categoría determinada. [5](#)

JFrog Artifactory Gestor universal de artefactos que almacena y gestiona paquetes, dependencias y artefactos binarios (imágenes Docker, paquetes Maven/NPM, etc.), facilitando el versionado y distribución en pipelines de CI/CD. [11](#)

Jira Herramienta de gestión ágil de proyectos que permite organizar tareas, historias de usuario y seguimiento de incidencias. [10](#), [12](#)

Kanban Metodología ágil de gestión visual del trabajo basada en tableros y tarjetas, que permite controlar el flujo de tareas en tiempo real. [10](#)

PostgreSQL Sistema de gestión de bases de datos relacional y de código abierto empleado para almacenar información estructurada. [10](#)

Postman Plataforma de colaboración para desarrollo de APIs que permite diseñar, probar, documentar y monitorizar servicios web mediante peticiones HTTP, colecciones de pruebas automatizadas y entornos configurables. [11](#)

Quay Registro de imágenes de contenedores (container registry) que permite almacenar, versionar y distribuir imágenes Docker en entornos empresariales. [11](#)

Retroplanning Técnica de planificación de proyectos que consiste en definir las tareas y hitos necesarios para alcanzar un objetivo, comenzando desde la fecha de finalización prevista y retrocediendo hasta el inicio del proyecto. [2](#)

SonarCloud Plataforma de análisis estático de código que ayuda a identificar problemas de calidad, vulnerabilidades y malas prácticas en el desarrollo de software. [11](#)

Spring Batch Framework de Spring orientado al procesamiento por lotes (*batch processing*), utilizado para ejecutar tareas asíncronas de larga duración. [6](#)